

UNIVERSIDADE CATÓLICA DE BRASÍLIA
CIÊNCIA DA COMPUTAÇÃO

JHONY MORAES DA SILVA
JOÃO VÍTOR FERREIRA CALADO
KAUÃ LACERDA DE LIMA
RAFAEL DE SOUSA SILVA

RELATÓRIO FINAL
SISTEMA GERENCIADOR DE LOJA - (SGL)

BRASÍLIA, DF
2026

JHONY MORAES DA SILVA
JOÃO VÍTOR FERREIRA CALADO
KAUÃ LACERDA DE LIMA
RAFAEL DE SOUSA SILVA

RELATÓRIO FINAL
SISTEMA GERENCIADOR DE LOJA - (SGL)

Documento apresentado a Universidade Católica de Brasília, campus Taguatinga, como requisito para aprovação bimestral parcial no curso de Teste de Software.

Orientador: Prof. Samuel Novais

BRASÍLIA, DF
2026

1.1 Finalidade

Este documento tem como finalidade apresentar a visão de alto nível do Sistema de Gerenciamento de Loja (SGL), descrevendo seus objetivos, escopo, usuários, necessidades, principais funcionalidades, restrições e requisitos gerais. O documento estabelece uma visão comum do produto antes da implementação das funcionalidades detalhadas.

1.2 Escopo

O SGL é u sistema web destinado ao gerenciamento das informações de uma loja. O sistema não realiza a operação de uma loja virtual nem funciona como marketplace. Assim, seu objetivo é fornecer ao vendedor uma área autenticada na qual ele possa cadastrar, consultar, alterar e excluir registros relacionados à sua operação, especialmente clientes, produtos e pedidos.

1.3 Definições, Acrônimos e Abreviações.

SGL –Sistema de Gerenciamento de Loja; **CRUD** –Create, Read, Update e Delete: criação, consulta, alteração e exclusão de dados; **API** – Interface de comunicação utilizada para permitir a interação entre componentes do sistema; **E2E** – End-to-End: teste de ponta a ponta realizado sobre o fluxo completo do sistema; **Frontend** – Camada de interface web utilizada pelo usuário; **Backend** – Camada responsável pelas regras de negócio, processamento de acesso aos dados; **Autenticação** – Processo de identificação do usuário para permitir acesso à sua conta.

1.4 Referências

DOCUMENTOS
HU01_Cadastro_de_Usuario
HU02_Autenticacao_do_Usuario
HU03_Cadastro_de_Registros
HU04_Consulta_de_Registros
HU05_Atualizacao_de_Registros
HU06_Exclusao_de_Registros

1.5 Visão Geral

O SGL será disponibilizado como aplicação web. A ideia é que o usuário realize autenticação para acessar sua conta e, a partir dela, possa gerenciar clientes, produtos e pedidos. O backend existente em Java será adaptado conforme as necessidades do projeto, enquanto o frontend será desenvolvido de forma simples utilizando HTML, CSS e JS.

A solução será validada em diferentes níveis, contemplando testes de unidade, testes de APIs/integração entre os componentes e, no final, testes E2E para verificar o comportamento do sistema completo.

2. Posicionamento

2.1 Oportunidade de Negócios

Pequenas lojas precisam ter um sistema de gerenciamento de dados de clientes, produtos e pedidos que seja organizado e seguro. O SGL busca atender a essa necessidade enquanto proporciona aos vendedores uma alternativa simples e focada nessa gestão, descentralizando os registros das planilhas, documentos de texto e outros arquivos armazenados em um único computador local com serviço de banco de dados.

2.2 Descrição do Problema

O problema de	dificuldade de manter clientes, produtos e pedidos organizados, atualizados e acessíveis de forma centralizada.
Afeta	vendedores e responsáveis por lojas que precisam consultar e atualizar informações da operação.
o impacto dele é	perda de organização, dificuldade de consulta, risco de inconsistência ou perda de registros e dependência do armazenamento local.

2.3 Sentença de Posição do Produto

Para	vendedores e responsáveis por lojas que necessitam organizar os dados da operação.
Quem	precisa cadastrar e consultar clientes, produtos e pedidos de maneira centralizada.
O SGL	é um sistema web de gerenciamento de loja.

Que	permite autenticação e operações de CRUD sobre clientes, produtos e pedidos, com armazenamento centralizado em banco de dados.
Nosso produto	oferece acesso autenticado aos registros do usuário por meio de uma aplicação web simples, com backend em Java e frontend web.

3. Descrições dos Envolvidos e Usuários

3.1 Demografia do Mercado

O público-alvo do SGL é formado por pessoas que possuem pequenas lojas e precisam gerenciar informações básicas do seu negócio de forma organizada e segura. Além disso, o sistema também é voltado a esses vendedores que buscam uma infraestrutura local simples, de maneira que o cenário ideal é o no qual o acesso aos dados por meio de uma conta autenticada é melhor do que manter anotações em documentos no computador do usuário.

3.2 Resumos dos envolvidos

ENVOLVIDOS		
NOME	DESCRIÇÃO	RESPONSABILIDADES
Analista / responsável pelo projeto	Responsável pelo levantamento e organização das necessidades do sistema.	Analisar requisitos, manter a visão do produto e orientar os ajustes necessários.
Desenvolvedor Backend	Responsável pela implementação existente em Java e seus aprimoramentos.	Manter regras de negócio, APIs, persistência e autenticação.
Desenvolvedor Frontend	Responsável pela interface web do sistema.	Implementar telas, formulários, navegação e comunicação com o backend.
Responsável por Banco de Dados	Responsável pela camada de persistência.	Garantir armazenamento, integridade e acesso adequado aos dados.

Responsável por Testes	Responsável pela validação do sistema.	Planejar e executar testes de unidade, API/integração e E2E.
------------------------	--	--

3.3 Resumos dos usuários

USUÁRIOS		
NOME	DESCRIÇÃO	RESPONSABILIDADES
Vendedor / Responsável pela loja	Usuário autenticado que possui uma conta no sistema.	Gerenciar seus clientes, produtos e pedidos; consultar e atualizar seus registros.

3.4 Ambiente do Usuário

O ambiente do usuário é o navegador, por onde o site hospedado será acessado. Lá o usuário pode criar sua conta e, a partir dela, começar a gerenciar (CRUD) os dados relacionados aos seus clientes, produtos e pedidos. O frontend da aplicação será desenvolvido em HTML, CSS, JavaScript e conectado ao backend em Java por meio de APIs disponíveis.

3.5 Perfis dos Interessados

INTERESSADO	DESCRIÇÃO	CRITÉRIO DE SUCESSO
Vendedor / Responsável pela loja	Principal interessado no uso do sistema para organização da operação.	Conseguir cadastrar, consultar, alterar e excluir seus registros com facilidade e segurança.
Equipe de desenvolvimento	Responsável por manter e evoluir a solução.	Entregar uma aplicação funcional, testável e sustentável.

3.6 Perfis dos Usuários

3.6.1 Vendedor / Responsável pela Loja

Representante	Vendedor ou responsável por uma loja.
Descrição	Usuário que cria e utiliza sua própria conta para gerenciar os dados relacionados à loja.
Responsabilidades	Autenticar-se no sistema; gerenciar clientes; gerenciar produtos; gerenciar pedidos; consultar e atualizar seus registros.

Critério de Sucesso	Realizar as operações necessárias de gerenciamento da loja sem acesso indevido aos dados de outras contas.
Envolvimento	Utilização recorrente do sistema para manutenção e consulta dos registros.

3.7 Principais Necessidades dos Usuários ou dos Envolvidos

NECESSIDADE	PRIORIDADE	PREOCUPAÇÕES	REALIDADE	SOLUÇÕES PROPOSTAS
Centralizar os dados da loja	Alta	Evitar perda e dispersão de informações.	Registros podem ficar espalhados em arquivos ou depender de um único computador.	Armazenar os dados em banco de dados e associá-los à conta autenticada do usuário.
Gerenciar clientes	Alta	Manter informações atualizadas.	Necessidade de consultar e alterar dados frequentemente.	CRUD de clientes por meio da aplicação web.
Gerenciar clientes	Alta	Manter catálogo e informações dos produtos organizados.	Atualizações manuais podem gerar inconsistências.	CRUD de produtos.
Gerenciar clientes	Alta	Registrar e consultar pedidos da loja.	Controle manual dificulta consultas e	CRUD de pedidos.

			manutenção do histórico.	
Garantir acesso seguro	Alta	Impedir acesso aos dados de outra conta.	Dados centralizados exigem controle de identidade e autorização.	Autenticação e associação dos registros ao usuário autenticado.

3.8 Alternativas e Competidores

Entre as alternativas estão planilhas eletrônicas, cadernos, arquivos locais e sistemas comerciais de gestão de lojas. O diferencial pretendido para o SGL é a simplicidade, com foco nas operações essenciais de cadastro e gerenciamento de clientes, produtos e pedidos, utilizando uma aplicação web e dados centralizados.

4. Visão Geral do Produto

4.1 Perspectiva do Produto

O SGL será uma aplicação web composta, de forma simplificada, por uma interface frontend, um backend em Java, um banco de dados para persistência e um mecanismo de autenticação. O frontend será responsável pela interação com o usuário e consumirá as APIs disponibilizadas pelo backend.

A solução deverá proporcionar:

- a) Acesso autenticado aos dados da conta do usuário;
- b) Persistência centralizada dos registros em banco de dados;
- c) Operações de CRUD para clientes, produtos e pedidos;
- d) Separação dos dados entre contas de usuários;
- e) Interface web simples e objetiva;
- f) Comunicação entre frontend e backend por meio de APIs;
- g) Possibilidade de evolução do backend existente sem necessidade de reescrever toda a solução.

4.2 Resumo dos Recursos

NECESSIDADES A SEREM ATENDIDAS	CARACTERÍSTICAS DO SEAD (REQUISITOS FUNCIONAIS)	BENEFÍCIOS ESPERADOS
Autenticação	Permitir criação/uso de conta e login para identificar o usuário.	Segurança e acesso individualizado.
Gerenciamento de clientes	Incluir, consultar, alterar e excluir clientes.	Organização dos dados dos clientes.
Gerenciamento de produtos	Incluir, consultar, alterar e excluir produtos.	Organização dos produtos da loja.
Gerenciamento de pedidos	Incluir, consultar, alterar e excluir pedidos.	Controle do histórico e dos pedidos.
Persistência	Armazenar os registros em banco de dados.	Disponibilidade dos dados sem depender do armazenamento local.
Integração frontend/backend	Consumir as APIs do backend a partir da aplicação web.	Separação de responsabilidades e facilidade de evolução.
Testes	Executar testes de unidade, testes de API/integração e testes E2E.	Maior confiabilidade e validação do sistema completo.

4.3 Custo e Preço

O custo e o preço comercial do produto é zero. Para fins acadêmicos e de desenvolvimento, serão priorizados recursos de software gratuitos e infraestrutura compatível com a proposta, considerando que o backend Java já existe e necessita principalmente de ajustes e melhorias. Caso o sistema seja disponibilizado em produção, deverão ser considerados custos de hospedagem da aplicação, banco de dados, domínio, monitoramento, manutenção e demais serviços de infraestrutura eventualmente utilizados.

4.4 Licenciamento e Instalação

A aplicação será disponibilizada como sistema web. O frontend poderá ser servido por um servidor web, enquanto o backend Java e o banco de dados serão executados em ambiente de

servidor. A instalação definitiva dependerá da infraestrutura escolhida para hospedagem. O licenciamento das tecnologias utilizadas deverá respeitar as respectivas licenças. Não há, nesta versão, uma definição de licença comercial específica para o SGL.

5. Restrições

5.1 Restrições do sistema

- a) O usuário depende de conectividade adequada para acessar a aplicação hospedada remotamente.
- b) O funcionamento depende da disponibilidade do backend e do banco de dados.
- c) O acesso aos registros deve ser condicionado à autenticação e às regras de autorização.
- d) O sistema não deve armazenar os dados principais da loja exclusivamente no computador do usuário.
- e) O frontend deve ser compatível com navegadores modernos.
- f) O backend existente em Java deve ser aproveitado sempre que possível, realizando-se os ajustes e melhorias necessários.
- g) As funcionalidades devem permanecer alinhadas às histórias de usuário já especificadas.

6. Outros Requisitos do Produto

6.1 Requisitos mínimos de Hardware

Como aplicação web, o sistema não exige hardware específico no computador do usuário além de

equipamento capaz de executar um navegador moderno. Computador, notebook ou dispositivo compatível com navegador web moderno.

Memória e processador suficientes para execução do navegador e uso normal da aplicação.

6.2 Requisitos mínimos de Software

Navegador web moderno com suporte a HTML, CSS e JavaScript.

Backend Java existente, com os ajustes necessários para disponibilização das APIs.

Sistema gerenciador de banco de dados utilizado pelo backend.

Ambiente de execução do backend Java compatível com a versão adotada no projeto.

7. Requisitos da Documentação

7.1 Manual do Usuário

O manual do usuário deverá apresentar orientações objetivas para utilização do sistema, permitindo que um vendedor consiga compreender o processo de autenticação e as operações de gerenciamento disponíveis.

7.1.1 Autenticação.

Criação ou utilização da conta.

Realização do login.

Acesso e encerramento da sessão.

Orientações básicas relacionadas à segurança da conta.

7.1.2 CRUD de cliente.

Cadastrar cliente.

Consultar clientes.

Alterar dados de cliente.

Excluir cliente.

7.1.3 CRUD de produto.

Cadastrar produto.

Consultar produtos.

Alterar dados de produto.

Excluir produto.

7.1.4 CRUD de pedido.

Cadastrar pedido.

Consultar pedidos.

Alterar dados de pedido.

Excluir pedido.

7.1.5 Ajuda Online

Quando aplicável, a documentação poderá disponibilizar orientações para dúvidas frequentes, erros de autenticação, uso das funcionalidades e procedimentos básicos para solução de problemas.

2 PLANO DE TESTE

2.1 INTRODUÇÃO E OBJETIVO DOS TESTES

O presente capítulo detalha o planejamento, o escopo operacional e a estratégia metodológica para a execução das atividades de teste no Sistema Gerenciador de Loja (SGL). O objetivo principal desta etapa é estabelecer critérios rigorosos de verificação e validação que assegurem a estabilidade, a integridade estrutural e a conformidade funcional do software perante os requisitos especificados.

O processo contempla uma pirâmide de testes bem definida, abrangendo testes unitários voltados para as validações de domínio nas classes de modelo, testes de integração e persistência focados na camada de acesso a dados (DAOs) via JDBC e transações atômicas, testes de desempenho e carga para avaliar o comportamento do sistema sob estresse utilizando o Apache JMeter, e testes de ponta a ponta (E2E) simulando o comportamento interativo do usuário na interface de linha de comando (MainCLI).

2.2 ESCOPO DO TESTE

2.2.1 O que será testado (In Scope):

Camada de Modelos e Regras de Negócio (Models): Validação estrita de restrições de domínio, garantindo que atributos críticos (como preços unitários, quantidades em estoque, formatação de documentos cadastrais e status permitidos para pedidos e pagamentos) rejeitem dados inconsistentes por meio do lançamento adequado de exceções.

Camada de Acesso a Dados e Persistência (DAOs): Verificação completa das operações de CRUD (Create, Read, Update, Delete) para todas as entidades principais do sistema: Clientes, Categorias, Produtos, Fornecedores e Vendedores, assegurando o correto mapeamento relacional via JDBC.

Transações Complexas e Integridade Atômica: Validação de fluxos transacionais complexos, tais como o método `inserirPedidoCompleto`, `atualizarPedidoCompleto` (com reversão lógica de estoque) e `deletarPedidoCompleto` (com reposição automática de itens ao inventário), controlando explicitamente o gerenciamento de transações com `setAutoCommit(false)`, `commit()` e `rollback()`.

Interface de Linha de Comando (MainCLI): Validação dos fluxos de navegação em menus e submenus, autenticação de operadores e tratamento de exceções decorrentes de entradas textuais inválidas inseridas pelo usuário no terminal.

Desempenho e Concorrência (Carga): Avaliação do comportamento da camada de persistência e conexões com o banco de dados sob cenários de requisições concorrentes simuladas por ferramentas de teste de carga.

2.2.2 O que NÃO será testado (Out of Scope):

- Desempenho e latência de infraestruturas de rede externa, provedores de hospedagem em nuvem ou instâncias físicas de servidores terceirizados.
- Mecanismos avançados de criptografia de senhas em infraestrutura de rede, visto que o escopo de segurança atual restringe-se à validação lógica local da aplicação CLI.
- Testes de compatibilidade gráfica ou visual, uma vez que o sistema opera predominantemente em ambiente de terminal (CLI), desconsiderando interfaces gráficas complexas de desktop nesta fase.

2.3 FERRAMENTAS UTILIZADAS

Para a execução e automação da qualidade no SGL, foram adotadas ferramentas padrão de mercado recomendadas para o ecossistema Java:

JUnit 5: Utilizado como framework principal para o desenvolvimento e execução de testes unitários isolados e testes de integração orientados a banco de dados. Permite o uso de anotações avançadas (`@Test`, `@BeforeEach`, `@ParameterizedTest`, `@DisplayName`) para estruturar cenários de sucesso e falha de forma limpa e organizada.

Apache JMeter: Empregado nos testes de desempenho, carga e estresse. Permite simular múltiplos usuários executando consultas e operações simultâneas na base de dados para mensurar o tempo de resposta e a estabilidade do SGL sob concorrência.

Drivers JDBC e Bancos de Dados de Teste: Conexões dedicadas a ambientes de homologação/teste para assegurar que a execução dos scripts de teste não corrompa os dados de produção da loja.

2.4 ESTRATÉGIA DE TESTES

2.4.1 Testes Unitários

Os testes unitários têm como foco isolar as classes de domínio e modelo (Cliente, Produto, Pagamento, Fornecedor, Categoria, entre outras) para validar suas regras de negócio de forma independente de banco de dados ou interfaces externas. O propósito é garantir que o sistema dispare exceções do tipo `IllegalArgumentException` ou equivalentes sempre que houver tentativa de inserção de valores inválidos (por exemplo, cadastro de um produto com preço

negativo, strings vazias em campos obrigatórios ou quantidades de estoque inválidas). Esses testes asseguram a consistência imediata do estado dos objetos logo em sua instanciação.

2.4.2 Testes de Integração e Camada de Dados (DAOs)

Esta estratégia visa validar a comunicação real e integrada entre as classes DAO e o banco de dados relacional. Serão testadas as operações de persistência e recuperação de registros para garantir que o mapeamento SQL executado via JDBC esteja correto e livre de falhas de sintaxe ou de mapeamento de tipos. Além disso, o foco central recai sobre os cenários transacionais: testa-se explicitamente se o uso de transações (rollback em caso de exceções e commit em caso de sucesso) protege o banco de dados contra estados corrompidos, como ocorreria caso uma venda fosse registrada sem saldo suficiente em estoque.

2.4.3 Testes de Carga e Desempenho (Apache JMeter)

Utilizando o Apache JMeter, a estratégia de testes de carga simula a concorrência de múltiplos usuários acessando rotinas de consulta de produtos, listagem de clientes e inserção de pedidos de forma simultânea. O objetivo é monitorar o comportamento do pool de conexões JDBC e identificar possíveis gargalos de desempenho ou travamentos na camada de persistência sob estresse operacional, garantindo que o SGL suporte picos de acesso típicos de um ambiente comercial real.

2.4.4 Testes End-to-End (E2E) e de Interface (CLI)

Os testes de ponta a ponta simulam a jornada completa do operador ou administrador de loja utilizando a interface de linha de comando (MainCLI). O fluxo compreende desde a inicialização do sistema, autenticação de credenciais de vendedor, navegação pelos menus de gerenciamento de produtos e fornecedores, até a execução completa de uma venda (criação do pedido, associação de itens, baixa no estoque e registro do pagamento). O propósito é validar a coesão entre todas as camadas da aplicação, desde a interação via terminal até a persistência definitiva das informações no banco de dados.

2.5 PAPÉIS E RESPONSABILIDADES (EXEMPLO PARA O GRUPO)

Para viabilizar a execução eficiente do plano de testes no contexto da equipe de desenvolvimento, as atribuições foram distribuídas da seguinte forma:

Desenvolvedor / Analista de Testes Unitários: Responsável por implementar, manter e executar os casos de teste em JUnit 5 voltados para a validação das classes de modelo e regras de domínio puras.

Responsável por Banco de Dados e DAOs: Encarregado de estruturar a massa de dados inicial (fixtures), configurar o ambiente de banco de dados de teste e validar a integridade dos testes de integração e transações complexas (Commit/Rollback).

Responsável por Testes de Carga (JMeter) e E2E: Encarregado de configurar os planos de teste de concorrência no JMeter, simular os fluxos de ponta a ponta na interface CLI e consolidar as métricas de desempenho e evidências de execução do sistema.

3 CASOS DE TESTE

1.1 CLASSE CATEGORIA (MODELS E DAO)

1.1.1 Casos de Teste da Camada Models (CategoriaTest)

CT-MOD-01: Alteração bem-sucedida do nome da categoria

- **Identificador:** CT-MOD-01
- **Descrição:** Validar se o método `setNome` atualiza corretamente o nome da categoria quando é passado um valor válido.
- **Passos para execução:**
 1. Instanciar um objeto `Categoria` com dados válidos.
 2. Chamar o método `instance.setNome("Categoria de roupas")`.
 3. Recuperar o nome com `instance.getNome()`.
- **Dados de Entrada:** "Categoria de roupas"
- **Saída Esperada:** O nome da categoria deve ser atualizado para "Categoria de roupas".
- **Critério de Sucesso:** O valor retornado por `getNome()` é idêntico ao valor enviado.

CT-MOD-02: Tentativa de definir nome vazio

- **Identificador:** CT-MOD-02
- **Descrição:** Validar se o sistema impede a atribuição de uma string vazia ao nome da categoria.
- **Passos para execução:**
 1. Instanciar um objeto `Categoria`.
 2. Chamar o método `instance.setNome("")` envolvendo a chamada em uma asserção de exceção.
- **Dados de Entrada:** "" (String vazia)

- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O nome da categoria não pode ser vazio."*
- **Critério de Sucesso:** A exceção é lançada e capturada corretamente pelo teste.

CT-MOD-03: Tentativa de definir nome nulo

- **Identificador:** CT-MOD-03
- **Descrição:** Validar se o sistema impede a atribuição de valor null ao nome da categoria.
- **Passos para execução:**
 1. Instanciar um objeto `Categoria`.
 2. Chamar o método `instance.setNome(null)`.
- **Dados de Entrada:** null
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O nome da categoria não pode ser vazio."*
- **Critério de Sucesso:** A exceção esperada é lançada.

CT-MOD-04: Tentativa de definir nome com mais de 100 caracteres

- **Identificador:** CT-MOD-04
- **Descrição:** Validar se o sistema restringe o tamanho máximo de 100 caracteres para o nome da categoria.
- **Passos para execução:**
 1. Instanciar um objeto `Categoria`.
 2. Gerar uma string com 101 caracteres (`"a".repeat(101)`).
 3. Chamar o método `instance.setNome(nomeGrande)`.
- **Dados de Entrada:** String com 101 caracteres (`'a' * 101`)
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O nome da categoria deve ter no máximo 100 caracteres."*
- **Critério de Sucesso:** A exceção de validação de tamanho máximo é disparada.

CT-MOD-05: Definição de nome com exatamente 100 caracteres (Limite)

- **Identificador:** CT-MOD-05
- **Descrição:** Validar se o sistema aceita um nome que possui exatamente o limite máximo permitido de 100 caracteres.
- **Passos para execução:**
 1. Instanciar um objeto `Categoria`.
 2. Gerar uma string com 100 caracteres (`"a".repeat(100)`).
 3. Chamar `instance.setNome(nome)`.
- **Dados de Entrada:** String com 100 caracteres (`'a' * 100`)

- **Saída Esperada:** O nome é aceito e armazenado com sucesso.
- **Critério de Sucesso:** getNome() retorna exatamente a string de 100 caracteres gerada.

CT-MOD-06: Definição de nome com espaços excedentes (Trim)

- **Identificador:** CT-MOD-06
- **Descrição:** Validar se o método remove automaticamente os espaços em branco nas pontas (trim).
- **Passos para execução:**
 1. Instanciar um objeto Categoria.
 2. Chamar instance.setNome(" Calçados ").
- **Dados de Entrada:** " Calçados "
- **Saída Esperada:** O nome deve ser armazenado sem os espaços excedentes.
- **Critério de Sucesso:** getNome() retorna "Calçados".

CT-MOD-07: Definição de descrição válida

- **Identificador:** CT-MOD-07
- **Descrição:** Validar a inserção de uma descrição válida para a categoria.
- **Passos para execução:**
 1. Instanciar um objeto Categoria.
 2. Chamar instance.setDescricao("Calçados").
- **Dados de Entrada:** "Calçados"
- **Saída Esperada:** Descrição atribuída com sucesso.
- **Critério de Sucesso:** getDescricao() retorna "Calçados".

CT-MOD-08: Definição de descrição nula

- **Identificador:** CT-MOD-08
- **Descrição:** Validar se o sistema aceita descrição nula (campo opcional).
- **Passos para execução:**
 1. Instanciar um objeto Categoria.
 2. Chamar instance.setDescricao(null).
- **Dados de Entrada:** null
- **Saída Esperada:** Descrição nula aceita sem erros.
- **Critério de Sucesso:** getDescricao() retorna null.

CT-MOD-09: Tentativa de descrição maior que 255 caracteres

- **Identificador:** CT-MOD-09
- **Descrição:** Validar a restrição de tamanho máximo de 255 caracteres para a descrição.
- **Passos para execução:**

1. Instanciar um objeto Categoria.
 2. Criar uma string de 256 caracteres ("a".repeat(256)).
 3. Chamar instance.setDescricao(descricaoGrande).
- **Dados de Entrada:** String com 256 caracteres
 - **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem *"A descrição deve ter no máximo 255 caracteres."*
 - **Critério de Sucesso:** Exceção lançada corretamente.

CT-MOD-10: Definição de descrição com exatamente 255 caracteres (Limite)

- **Identificador:** CT-MOD-10
- **Descrição:** Validar se o sistema aceita uma descrição com exatamente o limite de 255 caracteres.
- **Passos para execução:**
 1. Instanciar um objeto Categoria.
 2. Criar string de 255 caracteres ("a".repeat(255)).
 3. Chamar instance.setDescricao(descricao).
- **Dados de Entrada:** String com 255 caracteres
- **Saída Esperada:** Descrição aceita com sucesso.
- **Critério de Sucesso:** getDescricao() retorna a string completa de 255 caracteres.

CT-MOD-11: Atribuição de ID de Categoria Pai válido

- **Identificador:** CT-MOD-11
- **Descrição:** Validar a atribuição de um ID de categoria pai válido (inteiro positivo).
- **Passos para execução:**
 1. Instanciar um objeto Categoria.
 2. Chamar instance.setIdCategoriaPai(4).
- **Dados de Entrada:** 4
- **Saída Esperada:** ID de categoria pai definido com sucesso.
- **Critério de Sucesso:** getIdCategoriaPai() retorna 4.

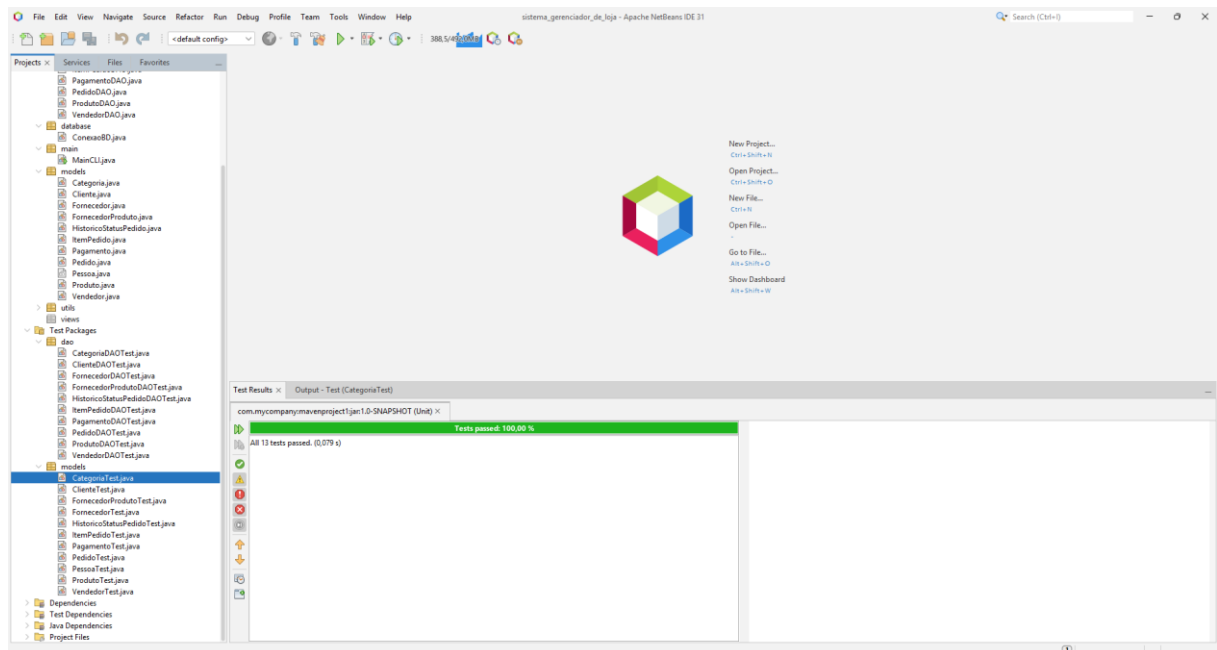
CT-MOD-12: Atribuição de ID de Categoria Pai nulo

- **Identificador:** CT-MOD-12
- **Descrição:** Validar que uma categoria pode não possuir categoria pai (valor null).
- **Passos para execução:**
 1. Instanciar um objeto Categoria.
 2. Chamar instance.setIdCategoriaPai(null).
- **Dados de Entrada:** null

- **Saída Esperada:** Atribuição aceita sem erros.
- **Critério de Sucesso:** getIdCategoriaPai() retorna null.

CT-MOD-13: Tentativa de ID de Categoria Pai negativo

- **Identificador:** CT-MOD-13
- **Descrição:** Validar se o sistema impede a atribuição de um ID de categoria pai negativo.
- **Passos para execução:**
 1. Instanciar um objeto Categoria.
 2. Chamar instance.setIdCategoriaPai(-1).
- **Dados de Entrada:** -1
- **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem *"O ID da categoria pai não pode ser negativo."*
- **Critério de Sucesso:** Exceção capturada com sucesso.



1.1.2 Casos de Teste da Camada DAO (CategoriaDAOTest)

CT-DAO-01: Inserção e Listagem por Vendedor

- **Identificador:** CT-DAO-01
- **Descrição:** Validar a persistência de uma nova categoria no banco de dados e a recuperação bem-sucedida através da listagem por ID de vendedor.
- **Passos para execução:**
 1. Configurar o ambiente (setUp), limpando o banco e inserindo um vendedor de teste válido para obter um idVendedorCadastrado.
 2. Instanciar um objeto Categoria preenchendo ID do vendedor, nome ("Eletrônicos"), descrição e ID pai como null.

3. Chamar categoriaDAO.inserir(categoria).
 4. Executar categoriaDAO.listarPorVendedor(idVendedorCadastrado).
- **Dados de Entrada:** Objeto Categoria com nome "Eletrônicos" vinculado ao vendedor cadastrado.
 - **Saída Esperada:**
 - O ID da categoria gerado pelo banco deve ser maior que 0 (categoria.getIdCategoria() > 0).
 - A lista retornada deve conter exatamente 1 elemento com os dados correspondentes.
 - **Critério de Sucesso:** Asserções de ID gerado positivo e tamanho/conteúdo da lista retornada executadas sem falhas.

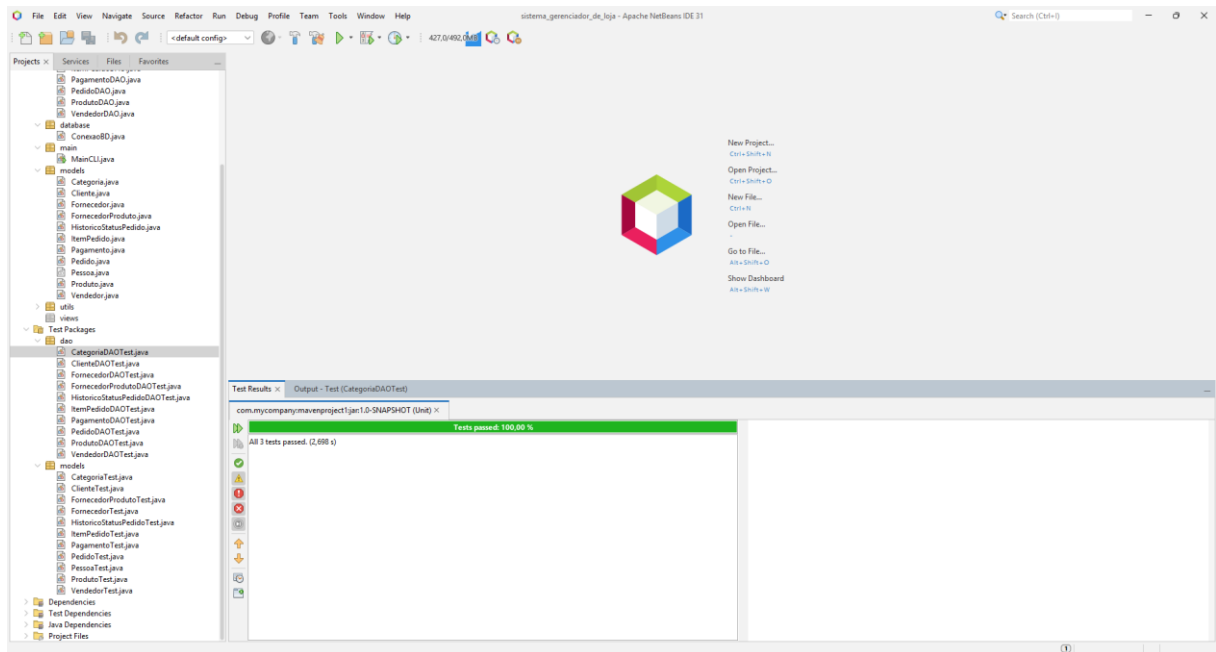
CT-DAO-02: Atualização de Categoria

- **Identificador:** CT-DAO-02
- **Descrição:** Validar a modificação de dados de uma categoria já cadastrada no banco de dados.
- **Passos para execução:**
 1. Inserir uma categoria inicial no banco (ex: Nome: "Roupas", Descrição: "Vestuário").
 2. Modificar as propriedades da instância de categoria para novos valores (Nome: "Moda e Vestuário", Descrição: "Roupas e acessórios").
 3. Chamar categoriaDAO.atualizar(categoria).
 4. Listar as categorias do vendedor para verificar a persistência da alteração.
- **Dados de Entrada:** Objeto Categoria com dados atualizados.
- **Saída Esperada:** Os dados atualizados devem refletir corretamente ao consultar o banco de dados.
- **Critério de Sucesso:** O nome e a descrição retornados na listagem correspondem exatamente aos novos valores alterados.

CT-DAO-03: Exclusão (Deleção) de Categoria

- **Identificador:** CT-DAO-03
- **Descrição:** Validar a remoção de uma categoria existente no banco de dados com base em seu ID e no ID do vendedor.
- **Passos para execução:**
 1. Inserir uma categoria de teste (ex: Nome: "Livros", Descrição: "Literatura").
 2. Capturar o idCategoria gerado.

3. Chamar `categoriaDAO.deletar(idCategoria, idVendedorCadastrado)`.
 4. Executar `categoriaDAO.listarPorVendedor(idVendedorCadastrado)`.
- **Dados de Entrada:** `idCategoria` e `idVendedorCadastrado`.
 - **Saída Esperada:** A categoria é removida do banco de dados, tornando a listagem vazia para aquele vendedor.
 - **Critério de Sucesso:** `categorias.isEmpty()` retorna verdadeiro (true).



CLASSE CLIENTE (MODELS E DAO)

1.2.1 Casos de Teste da Camada Models (ClienteTest)

CT-MOD-01: Definição de data de cadastro válida

- **Identificador:** CT-MOD-01
- **Descrição:** Validar se o método `setDataCadastro` atualiza corretamente a data quando é passada uma data/hora válida e passada.
- **Passos para execução:**
 1. Instanciar um objeto Cliente com dados válidos.
 2. Definir uma data válida específica (`LocalDateTime.of(2026, 9, 20, 14, 30)`).
 3. Chamar `instance.setDataCadastro(data)`.
 4. Recuperar com `instance.getDataCadastro()`.
- **Dados de Entrada:** 2026-09-20T14:30
- **Saída Esperada:** A data de cadastro deve ser atualizada exatamente para o valor informado.

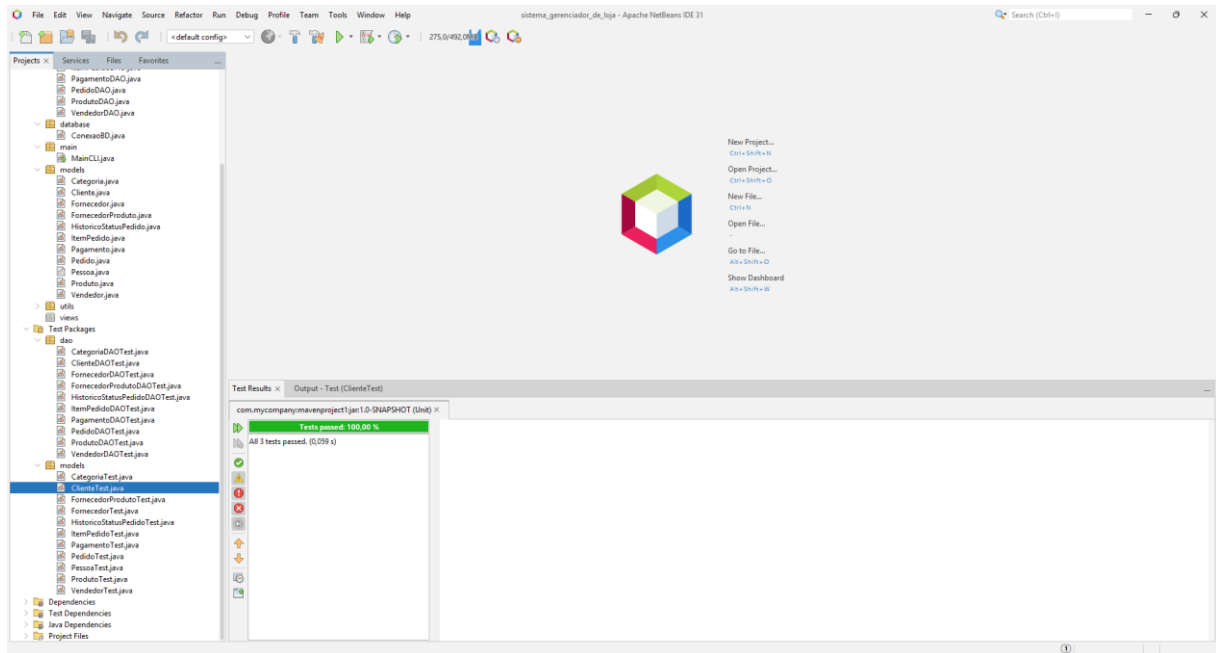
- **Critério de Sucesso:** O valor retornado por `getDataCadastro()` corresponde à data informada.

CT-MOD-02: Definição de data de cadastro nula (Padrão)

- **Identificador:** CT-MOD-02
- **Descrição:** Validar se ao definir a data de cadastro como nula, o sistema atribui automaticamente a data/hora atual (`LocalDateTime.now()`).
- **Passos para execução:**
 1. Instanciar um objeto Cliente.
 2. Chamar `instance.setDataCadastro(null)`.
 3. Verificar o valor retornado.
- **Dados de Entrada:** null
- **Saída Esperada:** O sistema não deve permitir valor nulo definitivo, atribuindo o momento atual.
- **Critério de Sucesso:** `getDataCadastro()` retorna um objeto `LocalDateTime` não nulo (`assertNotNull()`).

CT-MOD-03: Tentativa de definir data de cadastro no futuro

- **Identificador:** CT-MOD-03
- **Descrição:** Validar se o sistema impede a atribuição de uma data de cadastro situada no futuro.
- **Passos para execução:**
 1. Instanciar um objeto Cliente.
 2. Obter uma data no futuro (`LocalDateTime.now().plusDays(1)`).
 3. Chamar `instance.setDataCadastro(dataFutura)` envolvendo a chamada em uma asserção de exceção.
- **Dados de Entrada:** Data atual + 1 dia
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"A data de cadastro não pode ser no futuro."*
- **Critério de Sucesso:** A exceção é lançada e capturada corretamente pelo teste.



1.2.2 Casos de Teste da Camada DAO (ClienteDAOTest)

CT-DAO-01: Inserção bem-sucedida de Cliente

- **Identificador:** CT-DAO-01
- **Descrição:** Validar a persistência transacional de um novo cliente (envolvendo as tabelas pessoa e cliente) e a geração do ID correspondente.
- **Passos para execução:**
 1. Configurar o ambiente (setUp), limpando o banco e inserindo um vendedor de teste válido.
 2. Instanciar um objeto Cliente preenchendo nome, CPF, email, telefone, status ativo, ID do vendedor e data de cadastro.
 3. Chamar clienteDAO.inserir(cliente).
 4. Consultar os clientes cadastrados pelo ID do vendedor.
- **Dados de Entrada:** Objeto Cliente com dados completos vinculados ao vendedor de teste.
- **Saída Esperada:**
 - O ID da pessoa/cliente gerado deve ser maior que 0 (cliente.getIdPessoa() > 0).
 - A lista retornada não deve estar vazia e o nome do cliente deve conferir.
- **Critério de Sucesso:** O ID é gerado com sucesso e o cliente é encontrado na listagem do vendedor.

CT-DAO-02: Listagem de clientes por Vendedor

- **Identificador:** CT-DAO-02

- **Descrição:** Validar a recuperação da lista de clientes associados a um determinado vendedor por meio de junção (JOIN) entre as tabelas cliente e pessoa.
- **Passos para execução:**
 1. Inserir um cliente de teste vinculado ao vendedor de teste.
 2. Executar `clienteDAO.listarPorVendedor(vendedorTeste.getIdPessoa())`.
- **Dados de Entrada:** ID do vendedor cadastrado.
- **Saída Esperada:** Retorno de uma lista contendo exatamente o cliente inserido, com dados íntegros.
- **Critério de Sucesso:** O tamanho da lista é 1 e os dados recuperados coincidem com os inseridos.

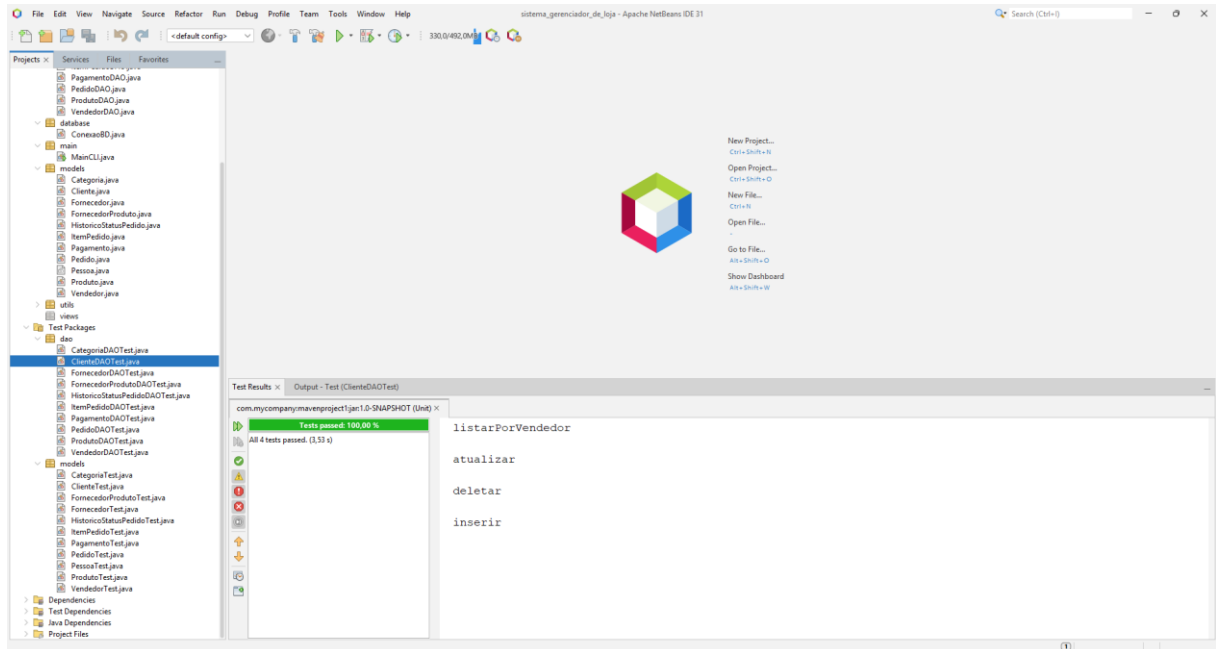
CT-DAO-03: Atualização de Cliente

- **Identificador:** CT-DAO-03
- **Descrição:** Validar a modificação dos dados cadastrais de um cliente existente (atualizando tabelas pessoa e cliente).
- **Passos para execução:**
 1. Inserir um cliente inicial (ex: Nome: "Cliente Antigo").
 2. Modificar as propriedades do objeto cliente (ex: Nome para "Cliente Atualizado" e novo e-mail).
 3. Chamar `clienteDAO.atualizar(cliente)`.
 4. Listar os clientes do vendedor para verificar as alterações.
- **Dados de Entrada:** Objeto Cliente com dados modificados.
- **Saída Esperada:** Os dados atualizados são gravados com sucesso no banco de dados.
- **Critério de Sucesso:** A listagem retorna o nome e o e-mail atualizados.

CT-DAO-04: Exclusão (Deleção) de Cliente

- **Identificador:** CT-DAO-04
- **Descrição:** Validar a remoção transacional de um cliente e de seu registro base na tabela pessoa.
- **Passos para execução:**
 1. Inserir um cliente de teste.
 2. Capturar o `id_pessoa` gerado.
 3. Chamar `clienteDAO.deletar(cliente.getIdPessoa(), vendedorTeste.getIdPessoa())`.
 4. Executar a listagem por vendedor para verificar a remoção.
- **Dados de Entrada:** ID do cliente e ID do vendedor.

- **Saída Esperada:** O registro é removido de ambas as tabelas (cliente e pessoa).
- **Critério de Sucesso:** A lista de clientes retornada para o vendedor fica vazia (`resultado.isEmpty()`).



CLASSE FORNECEDOR (MODELS E DAO)

1.3.1 Casos de Teste da Camada Models (FornecedorTest)

CT-MOD-01: Definição de CNPJ válido

- **Identificador:** CT-MOD-01
- **Descrição:** Validar se o método `setCnpj` atualiza corretamente o CNPJ quando é passado um valor válido de 14 dígitos.
- **Passos para execução:**
 1. Instanciar um objeto `Fornecedor` com dados válidos.
 2. Chamar `instance.setCnpj("12345678000191")`.
 3. Recuperar com `instance.getCnpj()`.
- **Dados de Entrada:** "12345678000191"
- **Saída Esperada:** O CNPJ deve ser atualizado para o valor informado.
- **Critério de Sucesso:** O valor retornado por `getCnpj()` é idêntico ao informado.

CT-MOD-02: Tentativa de definir CNPJ nulo

- **Identificador:** CT-MOD-02
- **Descrição:** Validar se o sistema impede a atribuição de valor nulo ao CNPJ.
- **Passos para execução:**
 1. Instanciar um objeto `Fornecedor`.

2. Chamar `instance.setCnpj(null)` envolvendo a chamada em uma asserção de exceção.

- **Dados de Entrada:** null
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O CNPJ não pode be nulo."* (ou correspondente).
- **Critério de Sucesso:** A exceção é lançada e capturada corretamente pelo teste.

CT-MOD-03: Tentativa de definir CNPJ com tamanho incorreto (diferente de 14 dígitos)

- **Identificador:** CT-MOD-03
- **Descrição:** Validar se o sistema restringe a quantidade exata de 14 dígitos para o CNPJ.
- **Passos para execução:**
 1. Instanciar um objeto `Fornecedor`.
 2. Chamar `instance.setCnpj("1234567890123")` (13 dígitos).
- **Dados de Entrada:** "1234567890123"
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O CNPJ deve conter exatamente 14 dígitos."*
- **Critério de Sucesso:** Exceção de validação de tamanho disparada.

CT-MOD-04: Definição de CNPJ formatado (Remoção de caracteres especiais)

- **Identificador:** CT-MOD-04
- **Descrição:** Validar se o método limpa a formatação (pontos, barras e traços) e armazena apenas os 14 dígitos numéricos do CNPJ.
- **Passos para execução:**
 1. Instanciar um objeto `Fornecedor`.
 2. Chamar `instance.setCnpj("12.345.678/9012-34")`.
- **Dados de Entrada:** "12.345.678/9012-34"
- **Saída Esperada:** O CNPJ limpo deve ser armazenado contendo apenas números.
- **Critério de Sucesso:** `getCnpj()` retorna "12345678901234".

CT-MOD-05: Definição de nome de fornecedor válido

- **Identificador:** CT-MOD-05
- **Descrição:** Validar a atribuição de um nome válido para o fornecedor.
- **Passos para execução:**
 1. Instanciar um objeto `Fornecedor`.
 2. Chamar `instance.setNome("Calçados")`.
- **Dados de Entrada:** "Calçados"
- **Saída Esperada:** Nome atribuído com sucesso.

- **Critério de Sucesso:** `getNome()` retorna "Calçados".

CT-MOD-06: Tentativa de definir nome nulo

- **Identificador:** CT-MOD-06
- **Descrição:** Validar se o sistema rejeita a atribuição de valor nulo ao nome do fornecedor.
- **Passos para execução:**
 1. Instanciar um objeto Fornecedor.
 2. Chamar `instance.setNome(null)`.
- **Dados de Entrada:** null
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O nome do fornecedor não pode ser vazio."*
- **Critério de Sucesso:** Exceção lançada corretamente.

CT-MOD-07: Tentativa de nome maior que 100 caracteres

- **Identificador:** CT-MOD-07
- **Descrição:** Validar o limite máximo de 100 caracteres para o nome do fornecedor.
- **Passos para execução:**
 1. Instanciar um objeto Fornecedor.
 2. Criar string de 101 caracteres (`"a".repeat(101)`).
 3. Chamar `instance.setNome(nomeGrande)`.
- **Dados de Entrada:** String com 101 caracteres
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O nome deve ter no máximo 100 caracteres."*
- **Critério de Sucesso:** Exceção disparada.

CT-MOD-08: Definição de nome com exatamente 100 caracteres (Limite)

- **Identificador:** CT-MOD-08
- **Descrição:** Validar se o sistema aceita um nome com exatamente 100 caracteres.
- **Passos para execução:**
 1. Instanciar um objeto Fornecedor.
 2. Criar string de 100 caracteres (`"a".repeat(100)`).
 3. Chamar `instance.setNome(nome)`.
- **Dados de Entrada:** String com 100 caracteres
- **Saída Esperada:** Nome aceito e armazenado com sucesso.
- **Critério de Sucesso:** `getNome()` retorna a string completa de 100 caracteres.

CT-MOD-09: Definição de nome com espaços excedentes (Trim)

- **Identificador:** CT-MOD-09

- **Descrição:** Validar a remoção automática de espaços em branco nas extremidades do nome.
- **Passos para execução:**
 1. Instanciar um objeto Fornecedor.
 2. Chamar `instance.setNome(" Calçados ")`.
- **Dados de Entrada:** " Calçados "
- **Saída Esperada:** Nome limpo de espaços externos.
- **Critério de Sucesso:** `getNome()` retorna "Calçados".

CT-MOD-10: Definição de e-mail válido

- **Identificador:** CT-MOD-10
- **Descrição:** Validar a inserção de um endereço de e-mail estruturalmente válido.
- **Passos para execução:**
 1. Instanciar um objeto Fornecedor.
 2. Chamar `instance.setEmail("teste@email.com")`.
- **Dados de Entrada:** "teste@email.com"
- **Saída Esperada:** E-mail atribuído com sucesso.
- **Critério de Sucesso:** `getEmail()` retorna "teste@email.com".

CT-MOD-11: Tentativa de definir e-mail nulo ou inválido

- **Identificador:** CT-MOD-11
- **Descrição:** Validar se o sistema rejeita e-mails nulos ou fora do padrão de formato válido (ex: sem @).
- **Passos para execução:**
 1. Instanciar um objeto Fornecedor.
 2. Chamar `instance.setEmail(null)` e posteriormente `instance.setEmail("emailinvalido.com")`.
- **Dados de Entrada:** null e "emailinvalido.com"
- **Saída Esperada:** Lançamento de `IllegalArgumentException` em ambas as tentativas.
- **Critério de Sucesso:** Exceção capturada adequadamente.

CT-MOD-12: Definição de e-mail com espaços excedentes (Trim)

- **Identificador:** CT-MOD-12
- **Descrição:** Validar a limpeza de espaços ao redor do e-mail.
- **Passos para execução:**
 1. Instanciar um objeto Fornecedor.
 2. Chamar `instance.setEmail(" teste@email.com ")`.

- **Dados de Entrada:** " teste@email.com "
- **Saída Esperada:** E-mail formatado sem espaços.
- **Critério de Sucesso:** getEmail() retorna "teste@email.com".

CT-MOD-13: Definição de telefone válido e formatado

- **Identificador:** CT-MOD-13
- **Descrição:** Validar a atribuição de telefones válidos (10 ou 11 dígitos), aceitando formatações com parênteses e traços.
- **Passos para execução:**
 1. Instanciar um objeto Fornecedor.
 2. Chamar instance.setTelefone("(61) 98888-7777").
- **Dados de Entrada:** "(61) 98888-7777"
- **Saída Esperada:** O telefone é limpo de caracteres não numéricos e armazenado com 11 dígitos.
- **Critério de Sucesso:** getTelefone() retorna "61988887777".

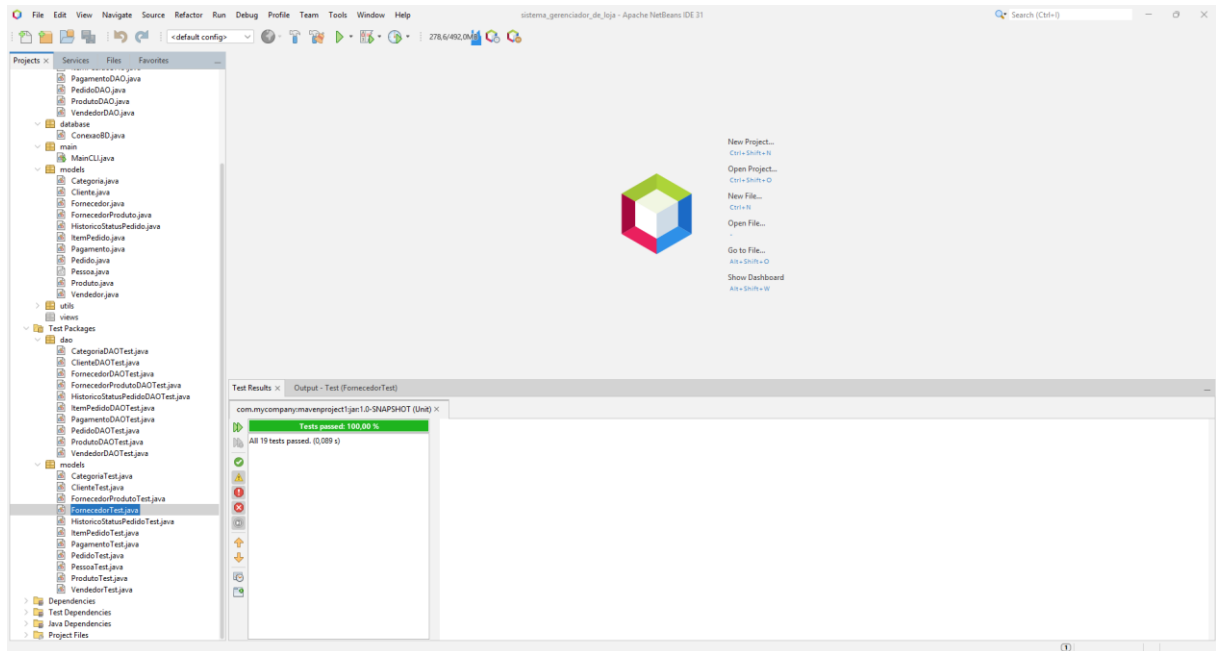
CT-MOD-14: Tentativa de telefone nulo ou vazio

- **Identificador:** CT-MOD-14
- **Descrição:** Validar que o sistema aceita telefone nulo ou vazio (campo opcional), convertendo-o internamente para null.
- **Passos para execução:**
 1. Instanciar um objeto Fornecedor.
 2. Chamar instance.setTelefone(null) e instance.setTelefone("").
- **Dados de Entrada:** null e ""
- **Saída Esperada:** Telefone definido como nulo sem gerar exceção.
- **Critério de Sucesso:** getTelefone() retorna null.

CT-MOD-15: Tentativa de telefone com tamanho inválido (> 11 dígitos)

- **Identificador:** CT-MOD-15
- **Descrição:** Validar se o sistema restringe o tamanho máximo de 11 dígitos numéricos para o telefone.
- **Passos para execução:**
 1. Instanciar um objeto Fornecedor.
 2. Chamar instance.setTelefone("2".repeat(12)).
- **Dados de Entrada:** String numérica com 12 dígitos
- **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem *"O telefone deve ter 10 ou 11 dígitos."*

- **Critério de Sucesso:** Exceção lançada corretamente.



1.3.2 Casos de Teste da Camada DAO (FornecedorDAOTest)

CT-DAO-01: Inserção bem-sucedida de Fornecedor

- **Identificador:** CT-DAO-01
- **Descrição:** Validar a persistência de um novo fornecedor no banco de dados e a geração automática do ID correspondente.
- **Passos para execução:**
 1. Configurar o ambiente (setUp), limpando o banco e cadastrando um vendedor de teste válido.
 2. Instanciar um objeto Fornecedor preenchendo ID do vendedor, CNPJ, nome, e-mail, telefone e status ativo.
 3. Chamar `fornecedorDAO.inserir(fornecedor)`.
- **Dados de Entrada:** Objeto Fornecedor preenchido com dados válidos vinculados ao vendedor.
- **Saída Esperada:** O ID do fornecedor gerado deve ser maior que 0 (`fornecedor.getIdFornecedor() > 0`).
- **Critério de Sucesso:** ID gerado e validado por asserção positiva.

CT-DAO-02: Listagem de Fornecedores por Vendedor

- **Identificador:** CT-DAO-02
- **Descrição:** Validar a recuperação da lista de fornecedores cadastrados por um vendedor específico.
- **Passos para execução:**

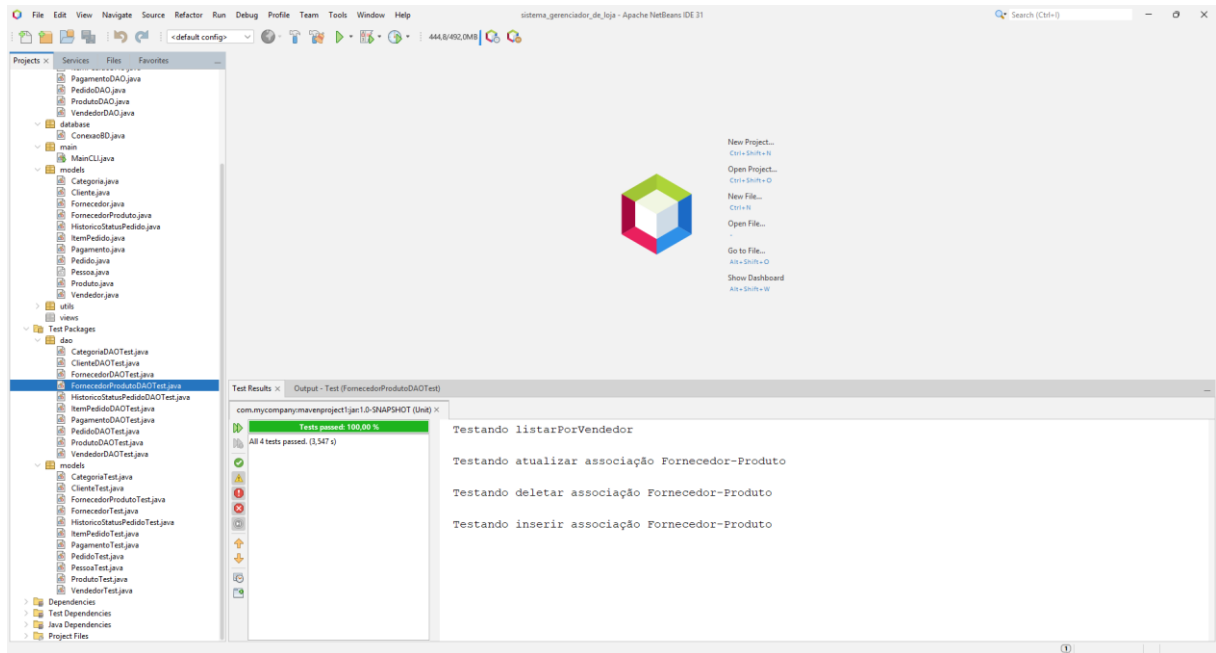
1. Inserir um fornecedor de teste vinculado ao vendedor de teste.
 2. Executar `fornecedorDAO.listarPorVendedor(vendedorTeste.getIdPessoa())`.
- **Dados de Entrada:** ID do vendedor cadastrado.
 - **Saída Esperada:** Retorno de uma lista contendo exatamente o fornecedor inserido, com dados íntegros.
 - **Critério de Sucesso:** A lista possui tamanho 1 e o nome corresponde a "Distribuidora Alpha".

CT-DAO-03: Atualização de Fornecedor

- **Identificador:** CT-DAO-03
- **Descrição:** Validar a alteração das informações cadastrais de um fornecedor já persistido.
- **Passos para execução:**
 1. Inserir um fornecedor inicial (ex: Nome: "Nome Antigo").
 2. Modificar as propriedades do objeto fornecedor para novos valores (Nome: "Nome Atualizado" e novo e-mail).
 3. Chamar `fornecedorDAO.atualizar(fornecedor)`.
 4. Listar os fornecedores do vendedor para validar a gravação.
- **Dados de Entrada:** Objeto Fornecedor com dados modificados.
- **Saída Esperada:** As alterações são salvas corretamente no banco de dados.
- **Critério de Sucesso:** A listagem reflete o novo nome e e-mail atualizados.

CT-DAO-04: Exclusão (Deleção) de Fornecedor

- **Identificador:** CT-DAO-04
- **Descrição:** Validar a remoção de um fornecedor do banco de dados com base em seu ID e ID do vendedor.
- **Passos para execução:**
 1. Inserir um fornecedor de teste.
 2. Capturar o `idFornecedor` gerado.
 3. Chamar `fornecedorDAO.deletar(fornecedor.getIdFornecedor(), vendedorTeste.getIdPessoa())`.
 4. Executar a listagem para verificar a exclusão.
- **Dados de Entrada:** ID do fornecedor e ID do vendedor.
- **Saída Esperada:** O registro do fornecedor é removido do banco de dados.
- **Critério de Sucesso:** A lista de fornecedores retornada fica vazia (`lista.isEmpty()`).



CLASSE FORNECEDOR-PRODUTO (MODELS E DAO)

1.4.1 Casos de Teste da Camada Models (FornecedorProdutoTest)

CT-MOD-01: Definição de preço de custo válido

- **Identificador:** CT-MOD-01
- **Descrição:** Validar se o método `setPrecoCusto` atualiza corretamente o preço quando é passado um valor positivo.
- **Passos para execução:**
 1. Instanciar um objeto `FornecedorProduto` com dados válidos.
 2. Chamar `instance.setPrecoCusto(56.90)`.
 3. Recuperar com `instance.getPrecoCusto()`.
- **Dados de Entrada:** 56.90
- **Saída Esperada:** O preço de custo deve ser atualizado para o valor informado.
- **Critério de Sucesso:** O valor retornado por `getPrecoCusto()` é idêntico ao informado (considerando margem de tolerância).

CT-MOD-02: Tentativa de definir preço de custo negativo

- **Identificador:** CT-MOD-02
- **Descrição:** Validar se o sistema impede a atribuição de valor negativo ao preço de custo.
- **Passos para execução:**
 1. Instanciar um objeto `FornecedorProduto`.
 2. Chamar `instance.setPrecoCusto(-89.90)` envolvendo a chamada em uma asserção de exceção.
- **Dados de Entrada:** -89.90

- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O preço de custo não pode ser negativo."*
- **Critério de Sucesso:** A exceção é lançada e capturada corretamente pelo teste.

CT-MOD-03: Definição de prazo de entrega em dias válido

- **Identificador:** CT-MOD-03
- **Descrição:** Validar se o método `setPrazoEntregaDias` aceita prazos superiores ou iguais a 1 dia.
- **Passos para execução:**
 1. Instanciar um objeto `FornecedorProduto`.
 2. Chamar `instance.setPrazoEntregaDias(5)`.
- **Dados de Entrada:** 5
- **Saída Esperada:** O prazo de entrega deve ser atualizado para o valor informado.
- **Critério de Sucesso:** `getPrazoEntregaDias()` retorna 5.

CT-MOD-04: Tentativa de definir prazo de entrega igual a zero

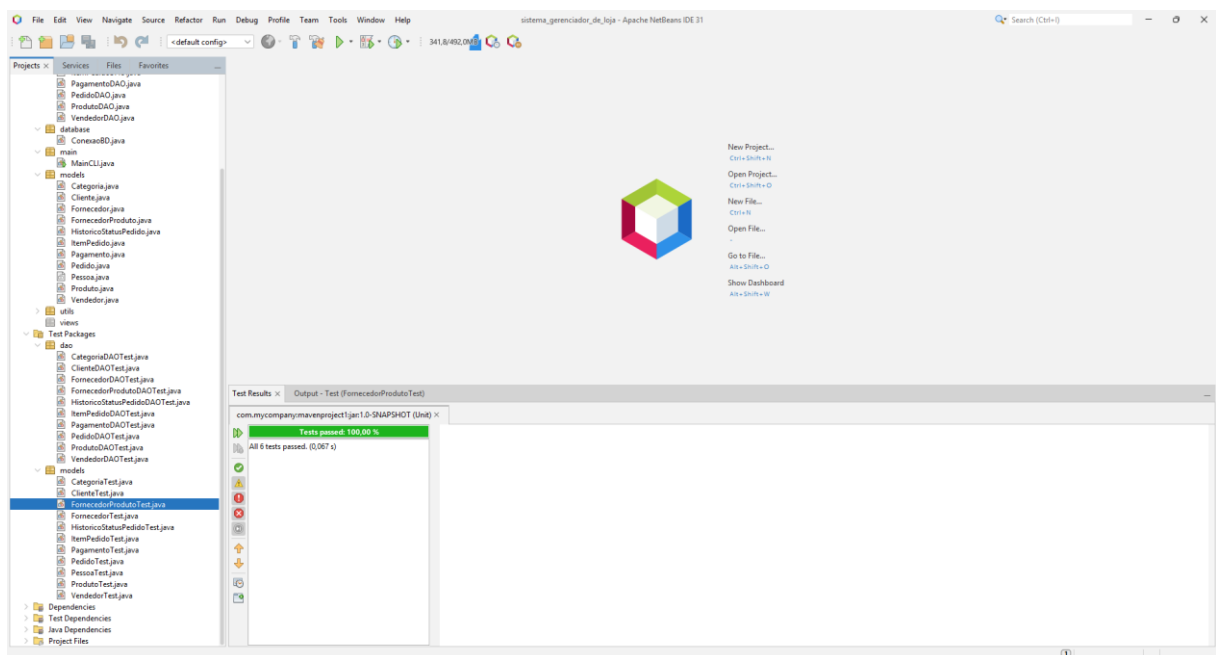
- **Identificador:** CT-MOD-04
- **Descrição:** Validar se o sistema restringe prazos de entrega iguais ou inferiores a zero.
- **Passos para execução:**
 1. Instanciar um objeto `FornecedorProduto`.
 2. Chamar `instance.setPrazoEntregaDias(0)`.
- **Dados de Entrada:** 0
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O prazo de entrega deve ser de pelo menos 1 dia."*
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-05: Definição de quantidade válida

- **Identificador:** CT-MOD-05
- **Descrição:** Validar a atribuição de uma quantidade estritamente positiva para a relação fornecedor-produto.
- **Passos para execução:**
 1. Instanciar um objeto `FornecedorProduto`.
 2. Chamar `instance.setQuantidade(25)`.
- **Dados de Entrada:** 25
- **Saída Esperada:** Quantidade atribuída com sucesso.
- **Critério de Sucesso:** `getQuantidade()` retorna 25.

CT-MOD-06: Tentativa de definir quantidade igual a zero

- **Identificador:** CT-MOD-06
- **Descrição:** Validar se o sistema rejeita quantidades iguais a zero.
- **Passos para execução:**
 1. Instanciar um objeto FornecedorProduto.
 2. Chamar instance.setQuantidade(0).
- **Dados de Entrada:** 0
- **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem "A quantidade deve ser maior do que zero."
- **Critério de Sucesso:** Exceção lançada corretamente.



1.4.2 Casos de Teste da Camada DAO (FornecedorProdutoDAOTest)

CT-DAO-01: Inserção bem-sucedida de Associação Fornecedor-Produto

- **Identificador:** CT-DAO-01
- **Descrição:** Validar a persistência de uma nova associação entre fornecedor e produto no banco de dados.
- **Passos para execução:**
 1. Configurar o ambiente (setUp), limpando o banco e cadastrando registros dependentes (vendedor, categoria, fornecedor e produto).
 2. Instanciar um objeto FornecedorProduto informando ID do fornecedor, ID do produto, ID do vendedor, preço de custo, prazo de entrega e quantidade.
 3. Chamar fornecedorProdutoDAO.inserir(fp).
- **Dados de Entrada:** Objeto FornecedorProduto preenchido com dados válidos.
- **Saída Esperada:** A inserção ocorre sem lançar exceções.

- **Critério de Sucesso:** A listagem por vendedor recupera 1 registro contendo exatamente os valores persistidos de preço, prazo e quantidade.

CT-DAO-02: Listagem de Associações por Vendedor

- **Identificador:** CT-DAO-02
- **Descrição:** Validar a recuperação correta da lista de associações de fornecedores e produtos vinculadas a um vendedor específico.
- **Passos para execução:**
 1. Inserir uma associação de teste vinculada ao vendedor.
 2. Executar `fornecedorProdutoDAO.listarPorVendedor(idVendedor)`.
- **Dados de Entrada:** ID do vendedor cadastrado.
- **Saída Esperada:** Retorno de uma lista contendo a associação inserida, com IDs de fornecedor e produto correspondentes.
- **Critério de Sucesso:** Lista não nula, com tamanho 1 e correspondência exata dos IDs.

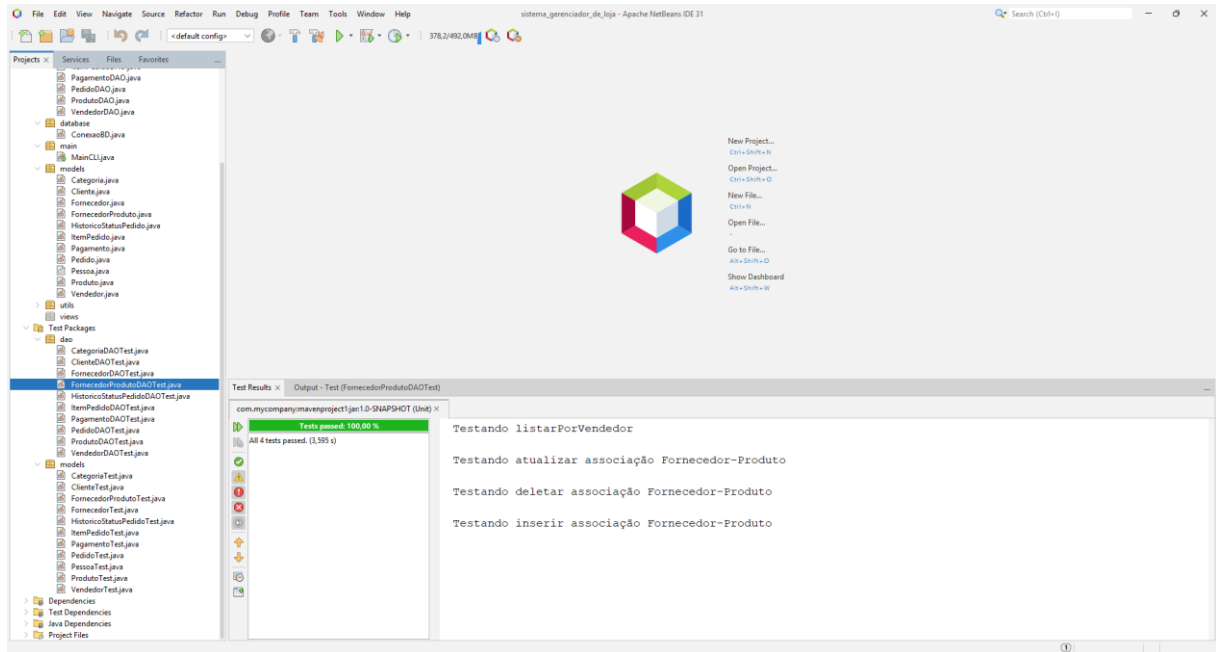
CT-DAO-03: Atualização de Associação Fornecedor-Produto

- **Identificador:** CT-DAO-03
- **Descrição:** Validar a alteração de atributos como preço de custo, prazo de entrega e quantidade de uma associação já persistida.
- **Passos para execução:**
 1. Inserir uma associação inicial (ex: Preço: 1000.00, Prazo: 7, Qtd: 10).
 2. Modificar as propriedades do objeto para novos valores (Preço: 950.00, Prazo: 4, Qtd: 25).
 3. Chamar `fornecedorProdutoDAO.atualizar(fp)`.
 4. Executar a listagem para validar a gravação.
- **Dados de Entrada:** Objeto `FornecedorProduto` com dados modificados.
- **Saída Esperada:** As alterações são salvas corretamente no banco de dados com base na chave composta (fornecedor, produto e vendedor).
- **Critério de Sucesso:** A listagem reflete o novo preço de custo, novo prazo e nova quantidade atualizados.

CT-DAO-04: Exclusão (Deleção) de Associação Fornecedor-Produto

- **Identificador:** CT-DAO-04
- **Descrição:** Validar a remoção de uma associação fornecedor-produto do banco de dados utilizando a chave composta.
- **Passos para execução:**
 1. Inserir uma associação de teste.

2. Chamar `fornecedorProdutoDAO.deletar(idFornecedor, idProduto, idVendedor)`.
 3. Executar a listagem para verificar a exclusão.
- **Dados de Entrada:** IDs de fornecedor, produto e vendedor.
 - **Saída Esperada:** O registro correspondente é removido do banco de dados.
 - **Critério de Sucesso:** A lista de associações retornada fica vazia (`lista.isEmpty()`).



CLASSE HISTÓRICO DE STATUS DO PEDIDO (MODELS E DAO)

1.5.1 Casos de Teste da Camada Models (HistoricoStatusPedidoTest)

CT-MOD-01: Definição de data de alteração válida

- **Identificador:** CT-MOD-01
- **Descrição:** Validar se o método `setDataAlteracao` atualiza corretamente a data e hora quando é passado um valor válido no passado ou presente.
- **Passos para execução:**
 1. Instanciar um objeto `HistoricoStatusPedido` com dados válidos.
 2. Chamar `instance.setDataAlteracao(LocalDateTime.of(2026, 5, 12, 14, 30))`.
 3. Recuperar com `instance.getDataAlteracao()`.
- **Dados de Entrada:** `LocalDateTime` correspondente a 2026-05-12 14:30
- **Saída Esperada:** A data de alteração deve ser atualizada para o valor informado.
- **Critério de Sucesso:** O valor retornado por `getDataAlteracao()` é idêntico ao informado.

CT-MOD-02: Definição de data de alteração nula (Atribuição de valor padrão)

- **Identificador:** CT-MOD-02
- **Descrição:** Validar se ao definir uma data nula, o sistema assume automaticamente a data e hora atuais (`LocalDateTime.now()`).

- **Passos para execução:**
 1. Instanciar um objeto `HistoricoStatusPedido`.
 2. Chamar `instance.setDataAlteracao(null)`.
- **Dados de Entrada:** null
- **Saída Esperada:** A data não deve ser nula, sendo preenchida com o momento atual.
- **Critério de Sucesso:** `getDataAlteracao()` retorna um objeto `LocalDateTime` válido e não nulo.

CT-MOD-03: Tentativa de definir data de alteração no futuro

- **Identificador:** CT-MOD-03
- **Descrição:** Validar se o sistema impede a atribuição de uma data de alteração situada no futuro.
- **Passos para execução:**
 1. Instanciar um objeto `HistoricoStatusPedido`.
 2. Definir uma data futura (ex: 2027-09-20 14:30) e chamar `instance.setDataAlteracao(dataFutura)` dentro de uma asserção de exceção.
- **Dados de Entrada:** `LocalDateTime` no futuro (2027-09-20 14:30)
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"A data de alteração não pode ser no futuro."*
- **Critério de Sucesso:** A exceção é lançada e capturada corretamente pelo teste.

CT-MOD-04: Definição de status válido

- **Identificador:** CT-MOD-04
- **Descrição:** Validar a atribuição de um status permitido (ex: "FINALIZADO").
- **Passos para execução:**
 1. Instanciar um objeto `HistoricoStatusPedido`.
 2. Chamar `instance.setStatus("FINALIZADO")`.
- **Dados de Entrada:** "FINALIZADO"
- **Saída Esperada:** Status atribuído com sucesso.
- **Critério de Sucesso:** `getStatus()` retorna "FINALIZADO".

CT-MOD-05: Tentativa de definir status nulo ou vazio

- **Identificador:** CT-MOD-05
- **Descrição:** Validar se o sistema rejeita valores nulos ou strings vazias para o status.
- **Passos para execução:**
 1. Instanciar um objeto `HistoricoStatusPedido`.
 2. Chamar `instance.setStatus(null)` e posteriormente `instance.setStatus("")`.

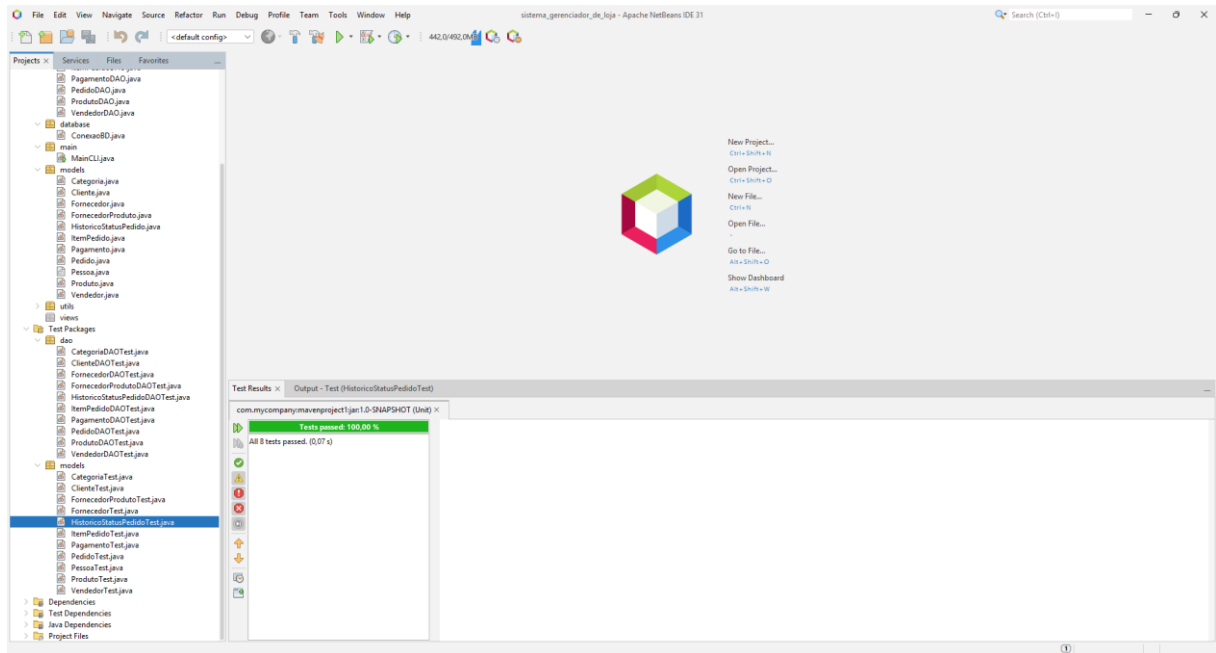
- **Dados de Entrada:** null e ""
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O status não pode ser nulo ou vazio."*
- **Critério de Sucesso:** Exceção capturada adequadamente.

CT-MOD-06: Definição de status em letras minúsculas (Normalização para Maiúsculas)

- **Identificador:** CT-MOD-06
- **Descrição:** Validar se o sistema converte e aceita status informados em letras minúsculas para o padrão em maiúsculas (ex: "pago" para "PAGO").
- **Passos para execução:**
 1. Instanciar um objeto `HistoricoStatusPedido`.
 2. Chamar `instance.setStatus("pago")`.
- **Dados de Entrada:** "pago"
- **Saída Esperada:** O status é aceito e convertido para letras maiúsculas.
- **Critério de Sucesso:** `getStatus()` retorna "PAGO".

CT-MOD-07: Tentativa de definir status inválido (Não permitido pelas regras de negócio)

- **Identificador:** CT-MOD-07
- **Descrição:** Validar se o sistema restringe status que não constam na lista permitida (PENDENTE, PAGO, FINALIZADO, CANCELADO).
- **Passos para execução:**
 1. Instanciar um objeto `HistoricoStatusPedido`.
 2. Chamar `instance.setStatus("EM_PROCESSAMENTO")`.
- **Dados de Entrada:** "EM_PROCESSAMENTO"
- **Saída Esperada:** Lançamento de `IllegalArgumentException` informando os status permitidos.
- **Critério de Sucesso:** Exceção disparada corretamente.



1.5.2 Casos de Teste da Camada DAO (HistoricoStatusPedidoDAOTest)

CT-DAO-01: Inserção e Listagem de Histórico por Pedido

- **Identificador:** CT-DAO-01
- **Descrição:** Validar a persistência de um registro de histórico de status vinculado a um pedido e a sua posterior recuperação por meio da listagem.
- **Passos para execução:**
 1. Configurar o ambiente (setUp), limpando o banco e cadastrando registros dependentes (vendedor, cliente e pedido).
 2. Instanciar um objeto HistoricoStatusPedido informando ID do pedido, data de alteração e status "PENDENTE".
 3. Chamar historicoDAO.inserir(historico).
 4. Executar historicoDAO.listarPorPedido(pedido.getIdPedido()).
- **Dados de Entrada:** Objeto HistoricoStatusPedido com dados válidos.
- **Saída Esperada:** Inserção bem-sucedida e retorno de uma lista contendo o histórico inserido ordenado por data.
- **Critério de Sucesso:** Lista não nula, com tamanho 1, contendo o ID do pedido correto e status igual a "PENDENTE".

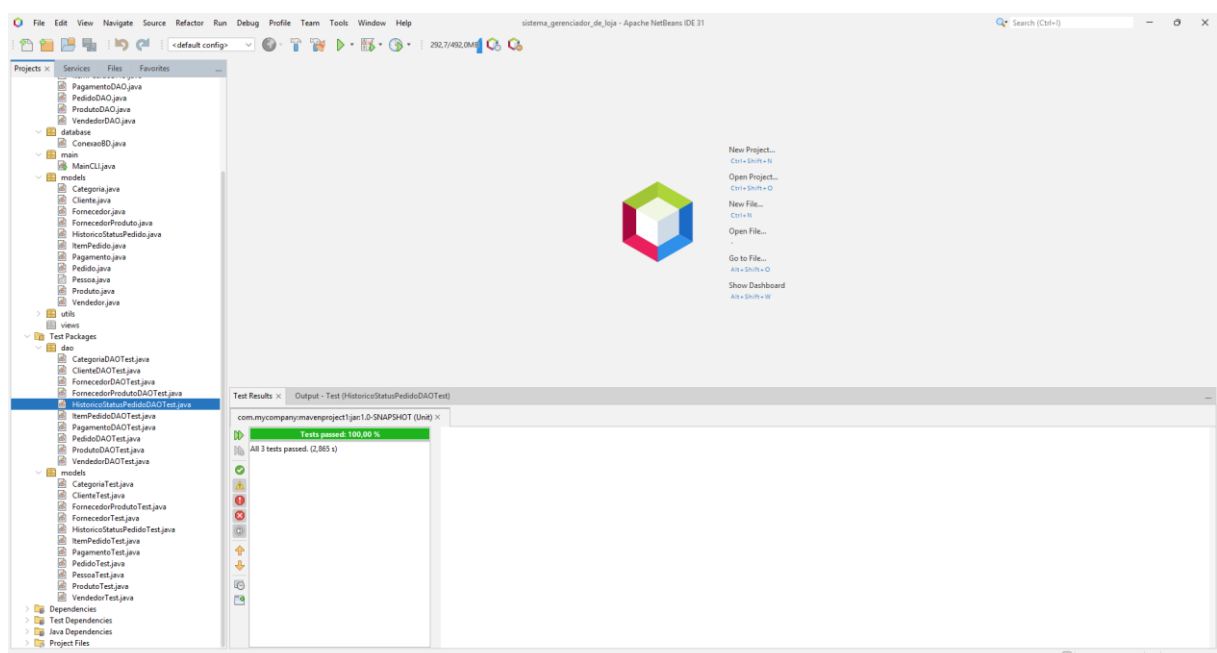
CT-DAO-02: Atualização de Histórico de Status

- **Identificador:** CT-DAO-02
- **Descrição:** Validar a alteração do status de um registro de histórico já persistido no banco de dados.
- **Passos para execução:**

1. Inserir um histórico inicial com status "PENDENTE".
 2. Recuperar o registro salvo da lista, alterar seu status para "PAGO" através de `setEstado/setStatus`.
 3. Chamar `historicoDAO.atualizar(historicoSalvo)`.
 4. Consultar novamente a listagem do pedido.
- **Dados de Entrada:** Objeto `HistoricoStatusPedido` com status modificado para "PAGO".
 - **Saída Esperada:** O status é atualizado com sucesso no banco de dados.
 - **Critério de Sucesso:** A listagem retorna o registro atualizado exibindo status "PAGO".

CT-DAO-03: Deleção de Histórico por Pedido

- **Identificador:** CT-DAO-03
- **Descrição:** Validar a remoção dos registros de histórico de status associados a um determinado pedido.
- **Passos para execução:**
 1. Inserir um histórico de status para o pedido de teste.
 2. Chamar `historicoDAO.deletarPorPedido(pedido.getIdPedido())`.
 3. Executar a listagem para verificar a exclusão.
- **Dados de Entrada:** ID do pedido correspondente.
- **Saída Esperada:** O histórico vinculado ao pedido é totalmente removido do banco.
- **Critério de Sucesso:** A lista de históricos retornada para o pedido fica vazia (`lista.isEmpty()`).



CLASSE ITEM DO PEDIDO (MODELS E DAO)

1.6.1 Casos de Teste da Camada Models (ItemPedidoTest)

CT-MOD-01: Definição de quantidade válida e recálculo de subtotal

- **Identificador:** CT-MOD-01
- **Descrição:** Validar se o método setQuantidade atualiza corretamente a quantidade do item e recalcula de forma automática o seu subtotal.
- **Passos para execução:**
 1. Instanciar um objeto ItemPedido com dados válidos.
 2. Chamar instance.setQuantidade(1).
 3. Verificar os métodos getQuantidade() e getSubtotal().
- **Dados de Entrada:** 1
- **Saída Esperada:** A quantidade é atualizada para 1 e o subtotal é recalculado adequadamente com base no preço unitário.
- **Critério de Sucesso:** A quantidade retornada é 1 e o subtotal corresponde ao novo cálculo esperado.

CT-MOD-02: Tentativa de definir quantidade igual a zero ou negativa

- **Identificador:** CT-MOD-02
- **Descrição:** Validar se o sistema impede a atribuição de quantidades menores ou iguais a zero para o item do pedido.
- **Passos para execução:**
 1. Instanciar um objeto ItemPedido.
 2. Chamar instance.setQuantidade(0) envolvendo a chamada em uma asserção de exceção.
- **Dados de Entrada:** 0
- **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem *"A quantidade do item deve ser maior que zero."*
- **Critério de Sucesso:** A exceção é lançada e capturada corretamente pelo teste.

CT-MOD-03: Definição de preço unitário válido e recálculo de subtotal

- **Identificador:** CT-MOD-03
- **Descrição:** Validar se a alteração do preço unitário atualiza corretamente o campo e recalcula o subtotal de forma automática.
- **Passos para execução:**
 1. Instanciar um objeto ItemPedido.
 2. Chamar instance.setPrecoUnitario(50.90).

3. Verificar `getPrecoUnitario()` e `getSubtotal()`.

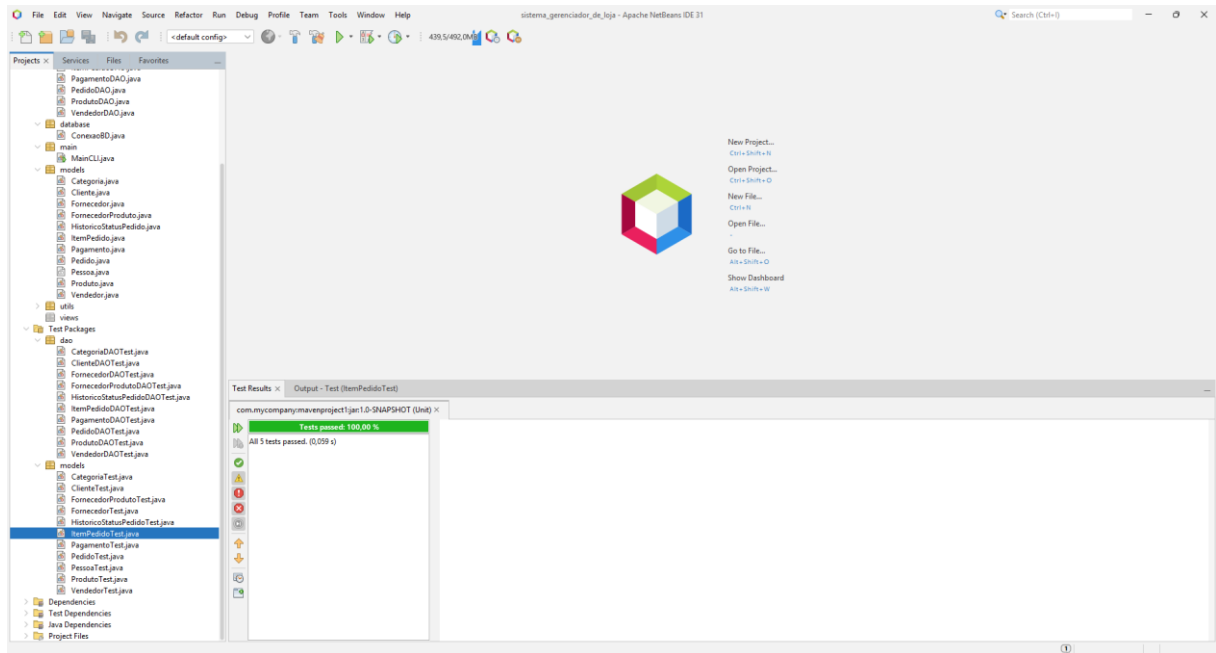
- **Dados de Entrada:** 50.90
- **Saída Esperada:** Preço unitário atualizado e subtotal recalculado multiplicando a quantidade pelo novo preço.
- **Critério de Sucesso:** O preço unitário e o subtotal correspondem aos valores esperados.

CT-MOD-04: Tentativa de definir preço unitário negativo

- **Identificador:** CT-MOD-04
- **Descrição:** Validar se o sistema rejeita a atribuição de preços unitários negativos.
- **Passos para execução:**
 1. Instanciar um objeto `ItemPedido`.
 2. Chamar `instance.setPrecoUnitario(-89.90)` dentro de uma asserção de exceção.
- **Dados de Entrada:** -89.90
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O preço unitário do item não pode ser negativo."*
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-05: Verificação do cálculo automático do subtotal inicial

- **Identificador:** CT-MOD-05
- **Descrição:** Validar se o construtor e as regras internas calculam o subtotal corretamente de acordo com a quantidade e o preço unitário informados.
- **Passos para execução:**
 1. Instanciar um objeto `ItemPedido` com quantidade 2 e preço unitário 89.90.
 2. Chamar `instance.getSubtotal()`.
- **Dados de Entrada:** Quantidade: 2, Preço Unitário: 89.90
- **Saída Esperada:** Retorno do subtotal com valor correspondente a 179.80.
- **Critério de Sucesso:** O subtotal retornado é igual a 179.80 (respeitando tolerância decimal).



1.6.2 Casos de Teste da Camada DAO (ItemPedidoDAOTest)

CT-DAO-01: Inserção e Listagem de Item do Pedido

- **Identificador:** CT-DAO-01
- **Descrição:** Validar a persistência de um novo item de pedido no banco de dados e a sua recuperação por meio da listagem por pedido.
- **Passos para execução:**
 1. Configurar o ambiente (setUp), inserindo dados dependentes (vendedor, cliente, categoria, produto e pedido).
 2. Instanciar um objeto ItemPedido informando ID do pedido, ID do produto, quantidade, preço unitário e subtotal calculado.
 3. Chamar itemPedidoDAO.inserir(item).
 4. Executar itemPedidoDAO.listarPorPedido(pedidoId).
- **Dados de Entrada:** Objeto ItemPedido preenchido com dados válidos.
- **Saída Esperada:** Inserção bem-sucedida e retorno de uma lista contendo o item inserido com os dados corretos.
- **Critério de Sucesso:** Lista não vazia com tamanho 1, contendo o ID do produto, quantidade e subtotal corretos.

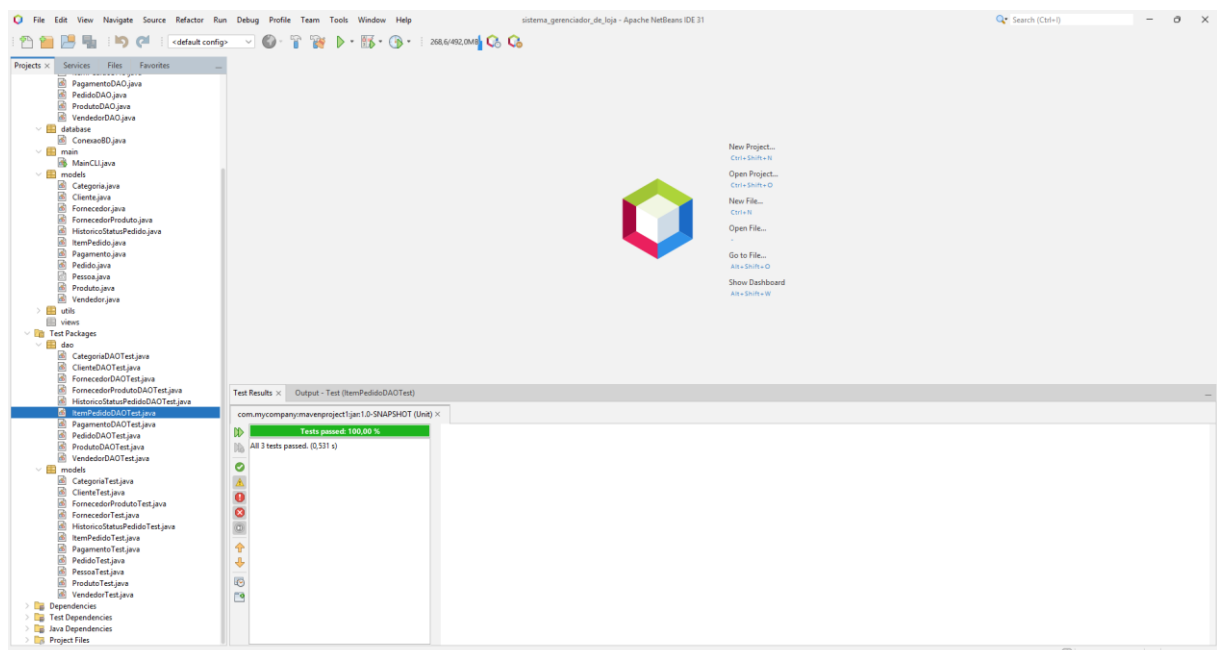
CT-DAO-02: Atualização de Item do Pedido

- **Identificador:** CT-DAO-02
- **Descrição:** Validar a alteração da quantidade ou valores de um item de pedido já persistido.
- **Passos para execução:**

1. Inserir um item de pedido inicial (ex: Quantidade: 1).
 2. Modificar a quantidade do item para um novo valor (ex: Quantidade: 3).
 3. Chamar `itemPedidoDAO.atualizar(item)`.
 4. Consultar a listagem do pedido para validar a gravação.
- **Dados de Entrada:** Objeto `ItemPedido` com quantidade modificada para 3.
 - **Saída Esperada:** A quantidade e o subtotal correspondente são atualizados no banco de dados.
 - **Critério de Sucesso:** A listagem reflete a nova quantidade (3) e o novo subtotal calculado.

CT-DAO-03: Deleção de Item do Pedido

- **Identificador:** CT-DAO-03
- **Descrição:** Validar a remoção de um item de pedido do banco de dados utilizando a chave composta (ID do pedido e ID do produto).
- **Passos para execução:**
 1. Inserir um item de pedido de teste.
 2. Chamar `itemPedidoDAO.deletar(pedidoId, produtoId)`.
 3. Executar a listagem para verificar a exclusão.
- **Dados de Entrada:** ID do pedido e ID do produto correspondentes.
- **Saída Esperada:** O item do pedido é removido do banco de dados.
- **Critério de Sucesso:** A lista de itens retornada para o pedido fica vazia (`items.isEmpty()`).



CLASSE PAGAMENTO (MODELS E DAO)

1.7.1 Casos de Teste da Camada Models (PagamentoTest)

CT-MOD-01: Definição de forma de pagamento válida

- **Identificador:** CT-MOD-01
- **Descrição:** Validar se o método setFormaPagamento aceita e armazena corretamente uma forma de pagamento permitida (ex: "DINHEIRO").
- **Passos para execução:**
 1. Instanciar um objeto Pagamento com dados válidos.
 2. Chamar instance.setFormaPagamento("DINHEIRO").
 3. Verificar com instance.getFormaPagamento().
- **Dados de Entrada:** "DINHEIRO"
- **Saída Esperada:** A forma de pagamento é atualizada com sucesso.
- **Critério de Sucesso:** getFormaPagamento() retorna "DINHEIRO".

CT-MOD-02: Tentativa de definir forma de pagamento nula ou vazia

- **Identificador:** CT-MOD-02
- **Descrição:** Validar se o sistema impede a atribuição de valores nulos ou vazios à forma de pagamento.
- **Passos para execução:**
 1. Instanciar um objeto Pagamento.
 2. Chamar instance.setFormaPagamento(null) e posteriormente instance.setFormaPagamento("") dentro de asserções de exceção.
- **Dados de Entrada:** null e ""
- **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem *"A forma de pagamento não pode ser vazia."*
- **Critério de Sucesso:** Exceção capturada adequadamente pelo teste.

CT-MOD-03: Definição de forma de pagamento em letras minúsculas (Normalização)

- **Identificador:** CT-MOD-03
- **Descrição:** Validar se o sistema normaliza e converte automaticamente a forma de pagamento informada em minúsculas para o padrão em maiúsculas (ex: "dinheiro" para "DINHEIRO").
- **Passos para execução:**
 1. Instanciar um objeto Pagamento.
 2. Chamar instance.setFormaPagamento("dinheiro").
- **Dados de Entrada:** "dinheiro"

- **Saída Esperada:** A forma de pagamento é aceita e convertida para letras maiúsculas.
- **Critério de Sucesso:** getFormaPagamento() retorna "DINHEIRO".

CT-MOD-04: Tentativa de definir forma de pagamento inválida

- **Identificador:** CT-MOD-04
- **Descrição:** Validar se o sistema rejeita formas de pagamento que não constam na lista permitida (DINHEIRO, CARTAO_CREDITO, CARTAO_DEBITO, PIX, BOLETO).
- **Passos para execução:**
 1. Instanciar um objeto Pagamento.
 2. Chamar instance.setFormaPagamento("VALE ALIMENTACAO") dentro de uma asserção de exceção.
- **Dados de Entrada:** "VALE ALIMENTACAO"
- **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem *"Forma de pagamento inválida."*
- **Critério de Sucesso:** Exceção disparada corretamente.

CT-MOD-05: Definição de data e hora de pagamento válida no passado

- **Identificador:** CT-MOD-05
- **Descrição:** Validar se o método setDataHora aceita corretamente uma data e hora válida situada no passado.
- **Passos para execução:**
 1. Instanciar um objeto Pagamento.
 2. Chamar instance.setDataHora(LocalDateTime.of(2026, 5, 11, 14, 30)).
- **Dados de Entrada:** LocalDateTime correspondente a 2026-05-11 14:30
- **Saída Esperada:** A data e hora do pagamento são atualizadas com sucesso.
- **Critério de Sucesso:** getDataHora() retorna o valor exato informado.

CT-MOD-06: Definição de data e hora nula (Atribuição de valor padrão)

- **Identificador:** CT-MOD-06
- **Descrição:** Validar se ao atribuir um valor nulo à data e hora, o sistema assume automaticamente o momento atual (LocalDateTime.now()).
- **Passos para execução:**
 1. Instanciar um objeto Pagamento.
 2. Chamar instance.setDataHora(null).
- **Dados de Entrada:** null
- **Saída Esperada:** A data não fica nula, sendo preenchida com o horário atual.

- **Critério de Sucesso:** `getDataHora()` retorna um objeto `LocalDateTime` válido e não nulo.

CT-MOD-07: Tentativa de definir data e hora no futuro

- **Identificador:** CT-MOD-07
- **Descrição:** Validar se o sistema impede o registro de uma data e hora de pagamento situada no futuro.
- **Passos para execução:**
 1. Instanciar um objeto `Pagamento`.
 2. Chamar `instance.setDataHora(LocalDateTime.of(2027, 3, 24, 14, 30))` dentro de uma asserção de exceção.
- **Dados de Entrada:** `LocalDateTime` no futuro (2027-03-24 14:30)
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"A data e hora do pagamento não podem estar no futuro."*
- **Critério de Sucesso:** Exceção capturada adequadamente.

CT-MOD-08: Definição de valor de pagamento válido

- **Identificador:** CT-MOD-08
- **Descrição:** Validar a atribuição de um valor monetário positivo ao pagamento (ex: 144.90).
- **Passos para execução:**
 1. Instanciar um objeto `Pagamento`.
 2. Chamar `instance.setValor(144.90)`.
- **Dados de Entrada:** 144.90
- **Saída Esperada:** O valor do pagamento é atualizado com sucesso.
- **Critério de Sucesso:** `getValor()` retorna 144.90.

CT-MOD-09: Tentativa de definir valor igual a zero ou negativo

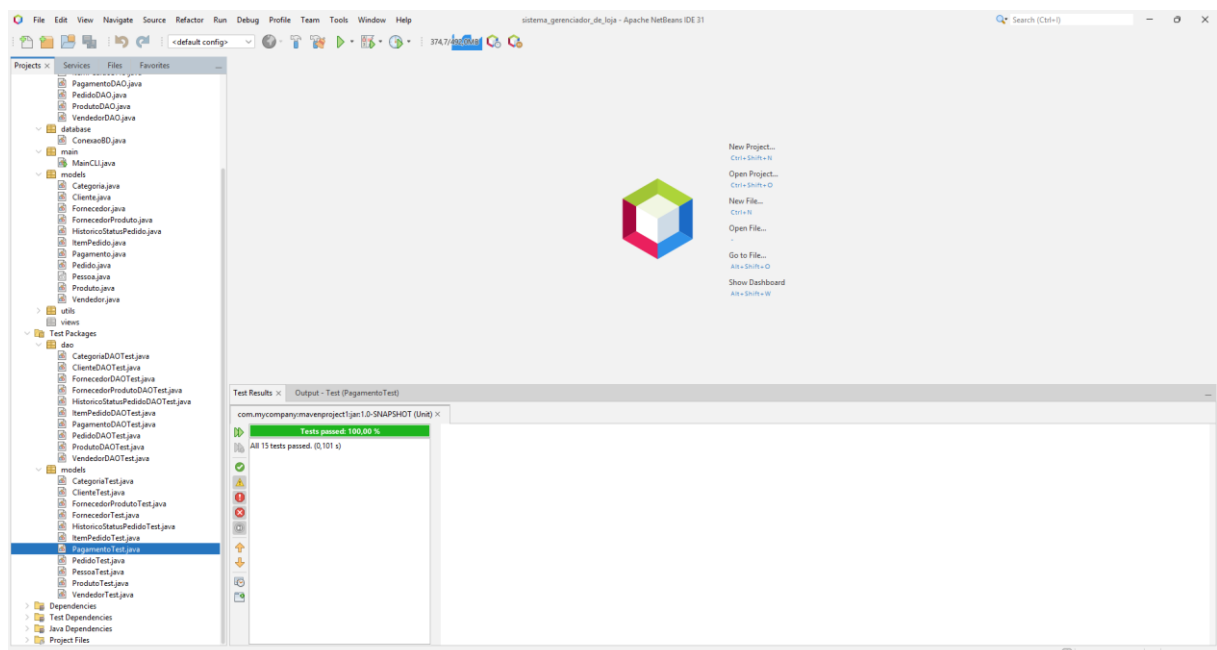
- **Identificador:** CT-MOD-09
- **Descrição:** Validar se o sistema rejeita valores de pagamento menores ou iguais a zero.
- **Passos para execução:**
 1. Instanciar um objeto `Pagamento`.
 2. Chamar `instance.setValor(0)` dentro de uma asserção de exceção.
- **Dados de Entrada:** 0
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O valor do pagamento deve ser maior que zero."*
- **Critério de Sucesso:** Exceção disparada corretamente.

CT-MOD-10: Definição de status de pagamento válido e normalização

- **Identificador:** CT-MOD-10
- **Descrição:** Validar se o sistema aceita e normaliza status permitidos (ex: "aprovado" para "APROVADO").
- **Passos para execução:**
 1. Instanciar um objeto Pagamento com status inicial "PROCESSANDO".
 2. Chamar `instance.setStatus("aprovado")`.
- **Dados de Entrada:** "aprovado"
- **Saída Esperada:** Status aceito e convertido para letras maiúsculas.
- **Critério de Sucesso:** `getStatus()` retorna "APROVADO".

CT-MOD-11: Tentativa de definir status de pagamento inválido, nulo ou vazio

- **Identificador:** CT-MOD-11
- **Descrição:** Validar se o sistema restringe status nulos, vazios ou que não pertençam à lista permitida (PROCESSANDO, APROVADO, RECUSADO, ESTORNADO).
- **Passos para execução:**
 1. Instanciar um objeto Pagamento.
 2. Chamar `instance.setStatus(null)`, `instance.setStatus("")` e `instance.setStatus("EM_ANDAMENTO")` dentro de asserções de exceção.
- **Dados de Entrada:** null, "", "EM_ANDAMENTO"
- **Saída Esperada:** Lançamento de `IllegalArgumentException` informando a restrição.
- **Critério de Sucesso:** Exceções capturadas corretamente em todas as tentativas.



1.7.2 Casos de Teste da Camada DAO (PagamentoDAOTest)

CT-DAO-01: Inserção de Pagamento com Geração de ID

- **Identificador:** CT-DAO-01
- **Descrição:** Validar a persistência de um novo registro de pagamento vinculado a um pedido e a geração automática do ID correspondente (id_pagamento).
- **Passos para execução:**
 1. Configurar o ambiente (setUp), limpando a base e inserindo dados dependentes necessários (vendedor, cliente, categoria, produto e pedido).
 2. Instanciar um objeto Pagamento informando ID do pedido, forma de pagamento ("PIX"), data/hora, valor e status ("APROVADO").
 3. Chamar pagamentoDAO.inserir(pagamento).
- **Dados de Entrada:** Objeto Pagamento preenchido com dados válidos.
- **Saída Esperada:** Inserção bem-sucedida no banco e preenchimento automático do ID gerado no objeto.
- **Critério de Sucesso:** pagamento.getIdPagamento() > 0 retorna verdadeiro.

CT-DAO-02: Listagem de Pagamentos

- **Identificador:** CT-DAO-02
- **Descrição:** Validar a recuperação de todos os pagamentos cadastrados no banco de dados por meio da listagem geral.
- **Passos para execução:**
 1. Inserir um registro de pagamento de teste no banco.
 2. Chamar pagamentoDAO.listar().
- **Dados de Entrada:** Nenhum (consulta à tabela de pagamentos).
- **Saída Esperada:** Retorno de uma lista contendo os registros de pagamento armazenados.
- **Critério de Sucesso:** A lista retornada não é nula e não está vazia (!lista.isEmpty()).

CT-DAO-03: Atualização de Pagamento

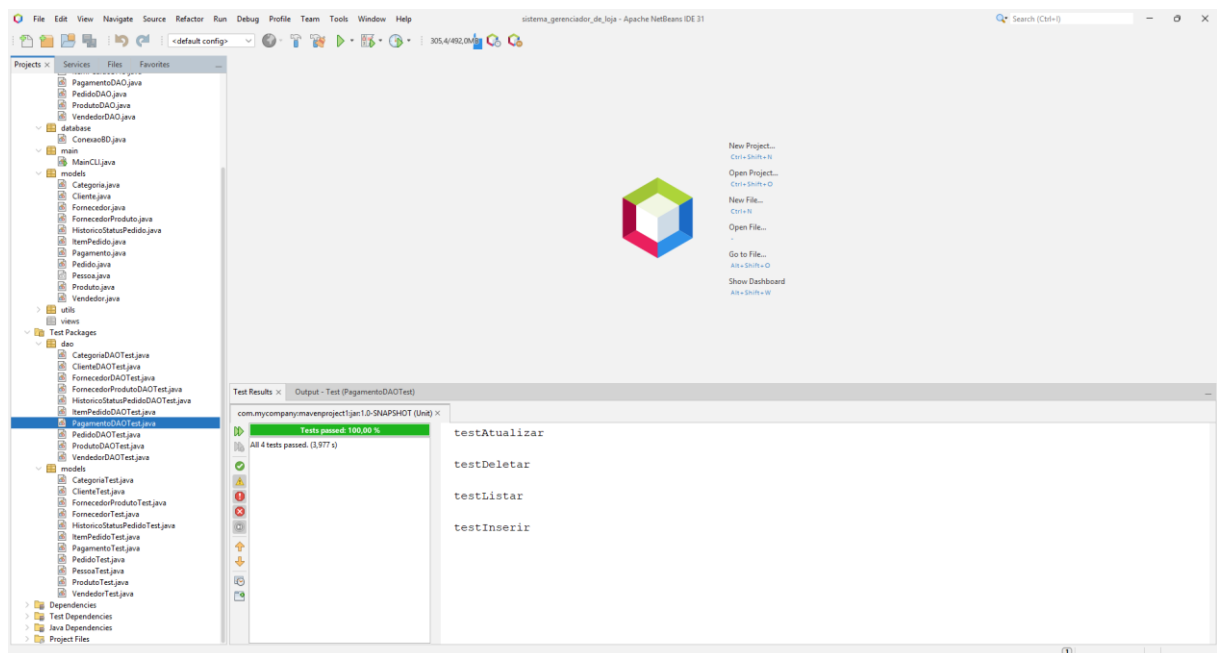
- **Identificador:** CT-DAO-03
- **Descrição:** Validar a modificação de dados (como o status) de um pagamento já persistido no banco de dados.
- **Passos para execução:**
 1. Inserir um pagamento com status "PROCESSANDO".
 2. Alterar o status do objeto para "APROVADO" e atualizar a data/hora.
 3. Chamar pagamentoDAO.atualizar(pagamento).

4. Executar `pagamentoDAO.listar()` para verificar a alteração.

- **Dados de Entrada:** Objeto `Pagamento` com o status modificado para "APROVADO".
- **Saída Esperada:** O registro é atualizado com sucesso na base de dados.
- **Critério de Sucesso:** A listagem confirma que o pagamento correspondente apresenta o novo status "APROVADO".

CT-DAO-04: Deleção de Pagamento

- **Identificador:** CT-DAO-04
- **Descrição:** Validar a remoção de um registro de pagamento do banco de dados com base no seu ID.
- **Passos para execução:**
 1. Inserir um pagamento de teste e capturar o seu ID gerado.
 2. Chamar `pagamentoDAO.deletar(idPagamento)`.
 3. Executar a listagem para confirmar a exclusão.
- **Dados de Entrada:** `idPagamento` correspondente.
- **Saída Esperada:** O registro de pagamento é completamente removido da base de dados.
- **Critério de Sucesso:** O pagamento deletado não é mais encontrado na listagem geral.



CLASSE PEDIDO (MODELS E DAO)

1.8.1 Casos de Teste da Camada Models (PedidoTest)

CT-MOD-01: Tentativa de instanciação com data de emissão no futuro (Cenário de Erro Proposal)

- **Identificador:** CT-MOD-01

- **Descrição:** Validar o comportamento do sistema quando é efetuada uma tentativa de criar um objeto Pedido utilizando uma data de emissão situada no futuro (evidenciando a falha/exceção por restrição de negócio).
- **Passos para execução:**
 1. Tentar instanciar um objeto Pedido passando diretamente uma data futura no construtor (ex: `LocalDateTime.of(2027, 9, 20, 14, 30)`).
- **Dados de Entrada:** `LocalDateTime` no futuro (2027-09-20 14:30)
- **Saída Esperada:** Lançamento de `IllegalArgumentException` pelo método `setDataEmissao`, impedindo a criação do objeto com data inválida.
- **Critério de Sucesso:** O teste demonstra o comportamento de rejeição de datas futuras (ou ilustra o erro proposital de configuração caso o teste tente prosseguir sem capturar a exceção).

CT-MOD-02: Definição de data de emissão válida

- **Identificador:** CT-MOD-02
- **Descrição:** Validar se o método `setDataEmissao` aceita e armazena corretamente uma data de emissão válida no passado ou presente.
- **Passos para execução:**
 1. Instanciar um objeto Pedido.
 2. Chamar `instance.setDataEmissao(LocalDate.of(2026, 4, 15, 14, 30))`.
- **Dados de Entrada:** `LocalDateTime` correspondente a 2026-04-15 14:30
- **Saída Esperada:** A data de emissão é atualizada com sucesso.
- **Critério de Sucesso:** `getDataEmissao()` retorna o valor exato informado.

CT-MOD-03: Definição de data de emissão nula (Atribuição de valor padrão)

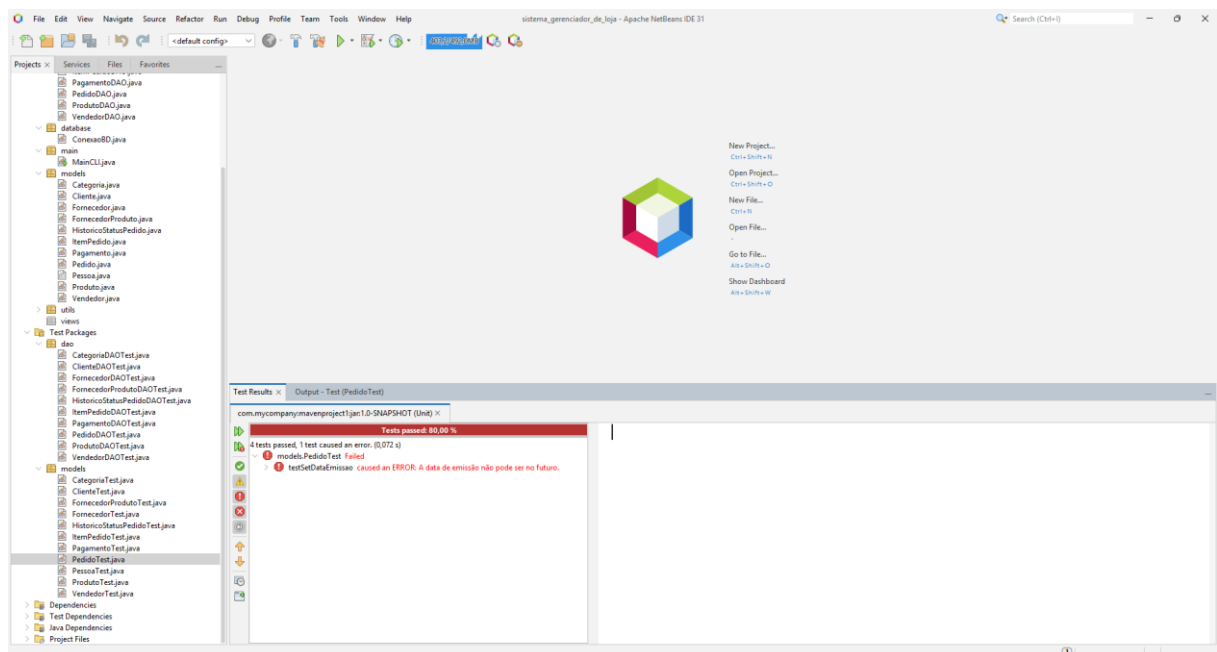
- **Identificador:** CT-MOD-03
- **Descrição:** Validar se ao atribuir um valor nulo à data de emissão, o sistema assume automaticamente o momento atual (`LocalDateTime.now()`).
- **Passos para execução:**
 1. Instanciar um objeto Pedido.
 2. Chamar `instance.setDataEmissao(null)`.
- **Dados de Entrada:** null
- **Saída Esperada:** A data não fica nula, sendo preenchida com o horário atual.
- **Critério de Sucesso:** `getDataEmissao()` retorna um objeto `LocalDateTime` válido e não nulo.

CT-MOD-04: Definição de valor total válido

- **Identificador:** CT-MOD-04
- **Descrição:** Validar a atribuição de um valor monetário positivo ao total do pedido (ex: 49.90).
- **Passos para execução:**
 1. Instanciar um objeto Pedido.
 2. Chamar `instance.setValorTotal(49.90)`.
- **Dados de Entrada:** 49.90
- **Saída Esperada:** O valor total do pedido é atualizado com sucesso.
- **Critério de Sucesso:** `getValorTotal()` retorna 49.90.

CT-MOD-05: Tentativa de definir valor total negativo

- **Identificador:** CT-MOD-05
- **Descrição:** Validar se o sistema rejeita a atribuição de valores totais negativos ao pedido.
- **Passos para execução:**
 1. Instanciar um objeto Pedido.
 2. Chamar `instance.setValorTotal(-179.80)` dentro de uma asserção de exceção.
- **Dados de Entrada:** -179.80
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O valor total do pedido não pode ser negativo."*
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.



1.8.2 Casos de Teste da Camada DAO (PedidoDAOTest)

CT-DAO-01: Inserção de Pedido Simples

- **Identificador:** CT-DAO-01
- **Descrição:** Validar a persistência de um novo pedido simples na base de dados e a geração automática do ID correspondente (id_pedido).
- **Passos para execução:**
 1. Configurar o ambiente (setUp), inserindo dados dependentes necessários (vendedor, cliente, categoria e produto).
 2. Instanciar um objeto Pedido associando o cliente e o vendedor de teste, definindo a data de emissão e o valor total.
 3. Chamar instance.inserir(pedido).
- **Dados de Entrada:** Objeto Pedido preenchido com dados válidos.
- **Saída Esperada:** Registro inserido com sucesso na base de dados e preenchimento do ID gerado.
- **Critério de Sucesso:** pedido.getIdPedido() > 0 retorna verdadeiro.

CT-DAO-02: Listagem de Pedidos por Vendedor

- **Identificador:** CT-DAO-02
- **Descrição:** Validar a recuperação dos pedidos associados a um determinado vendedor através da listagem específica.
- **Passos para execução:**
 1. Inserir um pedido de teste vinculado ao vendedor.
 2. Chamar instance.listarPorVendedor(vendedorTeste.getIdPessoa()).
- **Dados de Entrada:** ID do vendedor de teste.
- **Saída Esperada:** Retorno de uma lista contendo o pedido inserido.
- **Critério de Sucesso:** A lista não é nula, não está vazia e o tamanho retornado é igual a 1.

CT-DAO-03: Atualização de Pedido

- **Identificador:** CT-DAO-03
- **Descrição:** Validar a alteração de dados (como o valor total e data de emissão) de um pedido já persistido.
- **Passos para execução:**
 1. Inserir um pedido inicial com valor total de 100.0.
 2. Alterar o valor total para 150.0 e atualizar a data de emissão.
 3. Chamar instance.atualizar(pedido).
 4. Executar a listagem para confirmar a alteração.
- **Dados de Entrada:** Objeto Pedido com valor total modificado para 150.0.

- **Saída Esperada:** O registo é atualizado com sucesso na base de dados.
- **Critério de Sucesso:** A listagem confirma que o valor total associado é 150.0.

CT-DAO-04: Deleção de Pedido Simples

- **Identificador:** CT-DAO-04
- **Descrição:** Validar a remoção de um pedido simples da base de dados utilizando o ID do pedido e o ID do vendedor.
- **Passos para execução:**
 1. Inserir um pedido de teste.
 2. Chamar `instance.deletar(pedido.getIdPedido(), vendedorTeste.getIdPessoa())`.
 3. Consultar a listagem para verificar a exclusão.
- **Dados de Entrada:** ID do pedido e ID do vendedor.
- **Saída Esperada:** O pedido é completamente removido da base de dados.
- **Critério de Sucesso:** A lista de pedidos retornada para o vendedor fica vazia (`lista.isEmpty()`).

CT-DAO-05: Inserção de Pedido Completo (Transacional)

- **Identificador:** CT-DAO-05
- **Descrição:** Validar a operação transacional de inserção de um pedido completo, incluindo itens do pedido, baixa de stock, registo de pagamentos e histórico de status.
- **Passos para execução:**
 1. Preparar listas contendo um item de pedido, um pagamento e um registo de histórico de status.
 2. Chamar `instance.inserirPedidoCompleto(pedido, itens, pagamentos, historico)`.
- **Dados de Entrada:** Objeto Pedido, listas de ItemPedido, Pagamento e um objeto HistoricoStatusPedido.
- **Saída Esperada:** Transação concluída com sucesso (commit), gerando o ID do pedido e atualizando o stock do produto.
- **Critério de Sucesso:** `pedido.getIdPedido() > 0` e o pedido é listado corretamente.

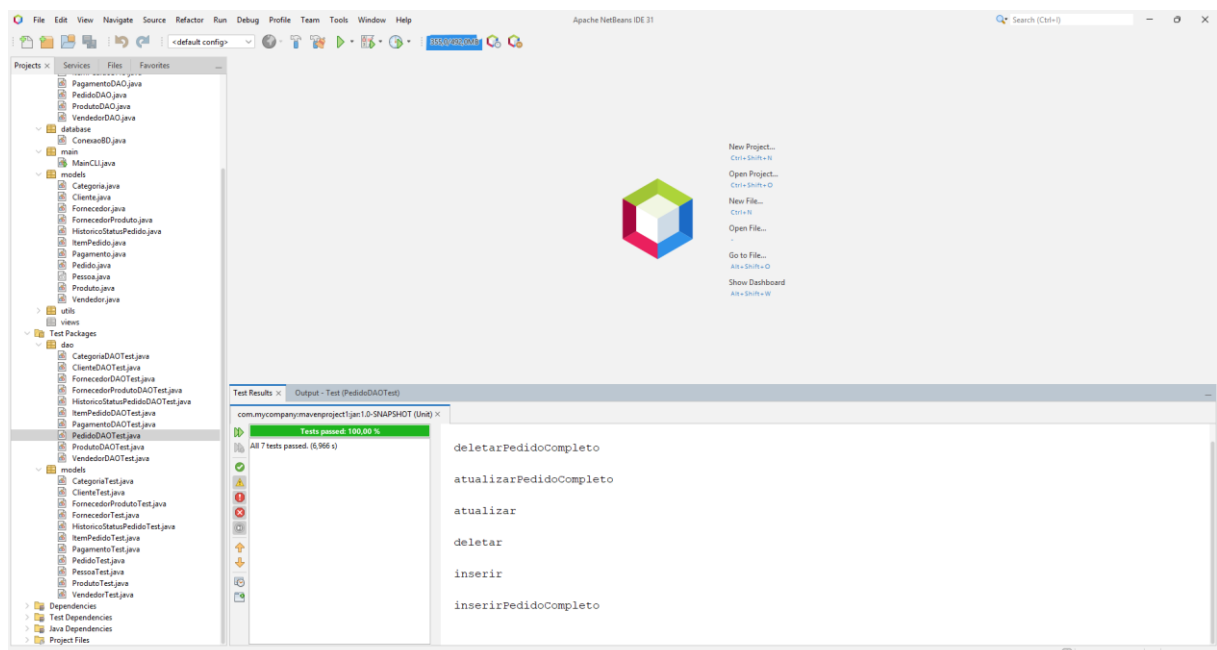
CT-DAO-06: Atualização de Pedido Completo (Transacional)

- **Identificador:** CT-DAO-06
- **Descrição:** Validar a atualização transacional de um pedido completo (substituição de itens, pagamentos e inclusão de novo histórico).
- **Passos para execução:**
 1. Inserir um pedido completo inicial.

2. Modificar os dados do pedido e criar um novo histórico de status (ex: de "PENDENTE" para "PAGO").
 3. Chamar `instance.atualizarPedidoCompleto(pedido, itens, pagamentos, novoHistorico)`.
- **Dados de Entrada:** Dados modificados do pedido, novos itens, novos pagamentos e novo histórico.
 - **Saída Esperada:** Atualização transacional efetivada com sucesso sem inconsistências.
 - **Critério de Sucesso:** A listagem reflete as alterações e o processo é executado sem exceções.

CT-DAO-07: Deleção de Pedido Completo (Transacional)

- **Identificador:** CT-DAO-07
- **Descrição:** Validar a exclusão transacional de um pedido completo, garantindo a remoção do histórico, pagamentos, itens, devolução do stock ao produto e eliminação do pedido.
- **Passos para execução:**
 1. Inserir um pedido completo de teste.
 2. Chamar `instance.deletarPedidoCompleto(pedido.getIdPedido(), vendedorTeste.getIdPessoa())`.
- **Dados de Entrada:** ID do pedido e ID do vendedor.
- **Saída Esperada:** Todos os registos associados são removidos e o stock é restituído.
- **Critério de Sucesso:** A listagem de pedidos do vendedor retorna vazia (`lista.isEmpty()`).



CLASSE PRODUTO (MODELS E DAO)

1.9.1 Casos de Teste da Camada Models (ProdutoTest)

CT-MOD-01: Definição de nome válido

- **Identificador:** CT-MOD-01
- **Descrição:** Validar se o método setNome aceita e armazena corretamente um nome de produto válido dentro dos limites permitidos.
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Chamar instance.setNome("Chuteira").
- **Dados de Entrada:** "Chuteira"
- **Saída Esperada:** O nome do produto é atualizado com sucesso.
- **Critério de Sucesso:** getNome() retorna "Chuteira".

CT-MOD-02: Tentativa de definir nome vazio

- **Identificador:** CT-MOD-02
- **Descrição:** Validar se o sistema rejeita a atribuição de uma string vazia como nome do produto.
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Chamar instance.setNome("") dentro de uma asserção de exceção.
- **Dados de Entrada:** "" (string vazia)
- **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem *"O nome do produto não pode be vazio."*
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-03: Tentativa de definir nome nulo

- **Identificador:** CT-MOD-03
- **Descrição:** Validar se o sistema impede a definição de um nome nulo para o produto.
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Chamar instance.setNome(null) dentro de uma asserção de exceção.
- **Dados de Entrada:** null
- **Saída Esperada:** Lançamento de IllegalArgumentException.
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-04: Tentativa de definir nome maior que 100 caracteres

- **Identificador:** CT-MOD-04

- **Descrição:** Validar se o sistema restringe o tamanho máximo do nome a 100 caracteres.
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Gerar uma string com 101 caracteres ("a".repeat(101)).
 3. Chamar instance.setNome(nomeGrande) dentro de uma asserção de exceção.
- **Dados de Entrada:** String com 101 caracteres ('a' repetido 101 vezes)
- **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem *"O nome do produto deve ter no máximo 100 caracteres."*
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-05: Definição de nome com limite exato de 100 caracteres

- **Identificador:** CT-MOD-05
- **Descrição:** Validar se o sistema aceita um nome que contenha exatamente o limite máximo permitido de 100 caracteres.
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Gerar uma string com 100 caracteres ("a".repeat(100)).
 3. Chamar instance.setNome(nome).
- **Dados de Entrada:** String com 100 caracteres
- **Saída Esperada:** O nome é aceito e armazenado sem exceções.
- **Critério de Sucesso:** getNome() retorna a string exata de 100 caracteres.

CT-MOD-06: Definição de nome com espaços vazios nas extremidades (Trim)

- **Identificador:** CT-MOD-06
- **Descrição:** Validar se o método remove automaticamente os espaços em branco excedentes nas extremidades do nome do produto.
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Chamar instance.setNome(" Tênis Esportivo ").
- **Dados de Entrada:** " Tênis Esportivo "
- **Saída Esperada:** O nome é formatado com o trim() aplicado.
- **Critério de Sucesso:** getNome() retorna "Tênis Esportivo" (sem os espaços extras).

CT-MOD-07: Definição de descrição válida

- **Identificador:** CT-MOD-07
- **Descrição:** Validar se o método setDescricao armazena corretamente uma descrição válida.

- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Chamar `instance.setDescricao("Chuteira masculina para futebol")`.
- **Dados de Entrada:** "Chuteira masculina para futebol"
- **Saída Esperada:** A descrição do produto é atualizada com sucesso.
- **Critério de Sucesso:** `getDescricao()` retorna a descrição informada.

CT-MOD-08: Definição de descrição nula

- **Identificador:** CT-MOD-08
- **Descrição:** Validar se o sistema permite atribuir um valor nulo à descrição do produto.
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Chamar `instance.setDescricao(null)`.
- **Dados de Entrada:** null
- **Saída Esperada:** A descrição passa a ser tratada como nula sem gerar falhas.
- **Critério de Sucesso:** `getDescricao()` retorna null.

CT-MOD-09: Tentativa de definir descrição maior que 255 caracteres

- **Identificador:** CT-MOD-09
- **Descrição:** Validar se o sistema restringe o tamanho máximo da descrição a 255 caracteres.
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Gerar uma string com 256 caracteres (`"a".repeat(256)`).
 3. Chamar `instance.setDescricao(descricaoGrande)` dentro de uma asserção de exceção.
- **Dados de Entrada:** String com 256 caracteres
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"A descrição do produto deve ter no máximo 255 caracteres."*
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-10: Definição de descrição com limite exato de 255 caracteres

- **Identificador:** CT-MOD-10
- **Descrição:** Validar se o sistema aceita uma descrição contendo exatamente 255 caracteres.
- **Passos para execução:**
 1. Instanciar um objeto Produto.

2. Gerar uma string com 255 caracteres ("a".repeat(255)).
 3. Chamar `instance.setDescricao(descricao)`.
- **Dados de Entrada:** String com 255 caracteres
 - **Saída Esperada:** A descrição é aceita e armazenada com sucesso.
 - **Critério de Sucesso:** `getDescricao()` retorna a string exata.

CT-MOD-11: Definição de preço unitário válido

- **Identificador:** CT-MOD-11
- **Descrição:** Validar a atribuição de um preço unitário positivo ao produto (ex: 126.90).
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Chamar `instance.setPrecoUnitario(126.90)`.
- **Dados de Entrada:** 126.90
- **Saída Esperada:** O preço unitário é atualizado com sucesso.
- **Critério de Sucesso:** `getPrecoUnitario()` retorna 126.90 (com tolerância de precisão).

CT-MOD-12: Tentativa de definir preço unitário negativo

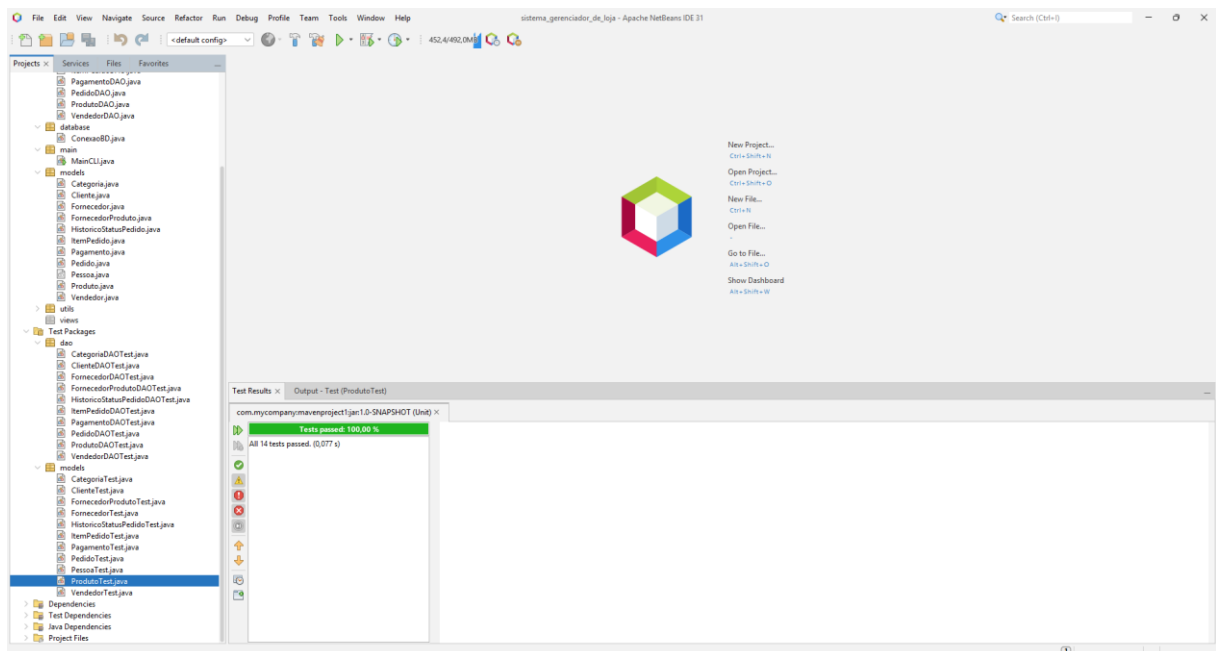
- **Identificador:** CT-MOD-12
- **Descrição:** Validar se o sistema rejeita a atribuição de um preço unitário negativo.
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Chamar `instance.setPrecoUnitario(-199.90)` dentro de uma asserção de exceção.
- **Dados de Entrada:** -199.90
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O preço unitário não pode ser negativo."*
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-13: Definição de estoque válido

- **Identificador:** CT-MOD-13
- **Descrição:** Validar a atribuição de uma quantidade de estoque válida e não negativa (ex: 30).
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Chamar `instance.setEstoque(30)`.
- **Dados de Entrada:** 30
- **Saída Esperada:** O estoque é atualizado com sucesso.
- **Critério de Sucesso:** `getEstoque()` retorna 30.

CT-MOD-14: Tentativa de definir estoque negativo

- **Identificador:** CT-MOD-14
- **Descrição:** Validar se o sistema rejeita a configuração de valores negativos para o estoque.
- **Passos para execução:**
 1. Instanciar um objeto Produto.
 2. Chamar instance.setEstoque(-1) dentro de uma asserção de exceção.
- **Dados de Entrada:** -1
- **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem "O estoque não pode ser negativo."
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.



1.9.2 Casos de Teste da Camada DAO (ProdutoDAOTest)

CT-DAO-01: Inserção de Produto

- **Identificador:** CT-DAO-01
- **Descrição:** Validar a persistência de um novo produto na base de dados, a geração automática do ID correspondente (id_produto) e a recuperação via listagem.
- **Passos para execução:**
 1. Configurar o ambiente (setUp), inserindo dados de suporte necessários (vendedor e categoria de teste).
 2. Instanciar um objeto Produto associando o ID do vendedor e da categoria, definindo nome, descrição, preço unitário, estoque e status ativo.
 3. Chamar produtoDAO.inserir(produto).

- **Dados de Entrada:** Objeto Produto preenchido com dados válidos (ex: "Smartphone X").
- **Saída Esperada:** Registro inserido com sucesso na base de dados, gerando um ID válido.
- **Critério de Sucesso:** `produto.getIdProduto() > 0` e o produto consta na listagem por vendedor com o nome correto.

CT-DAO-02: Listagem de Produtos por Vendedor

- **Identificador:** CT-DAO-02
- **Descrição:** Validar a recuperação de todos os produtos associados a um determinado vendedor através da listagem específica.
- **Passos para execução:**
 1. Inserir um produto de teste (ex: "Notebook Y") vinculado ao vendedor de teste.
 2. Chamar `produtoDAO.listarPorVendedor(vendedorTeste.getIdPessoa())`.
- **Dados de Entrada:** ID do vendedor de teste.
- **Saída Esperada:** Retorno de uma lista contendo exatamente o produto inserido.
- **Critério de Sucesso:** A lista não é nula, contém tamanho igual a 1 e o nome do produto retornado corresponde a "Notebook Y".

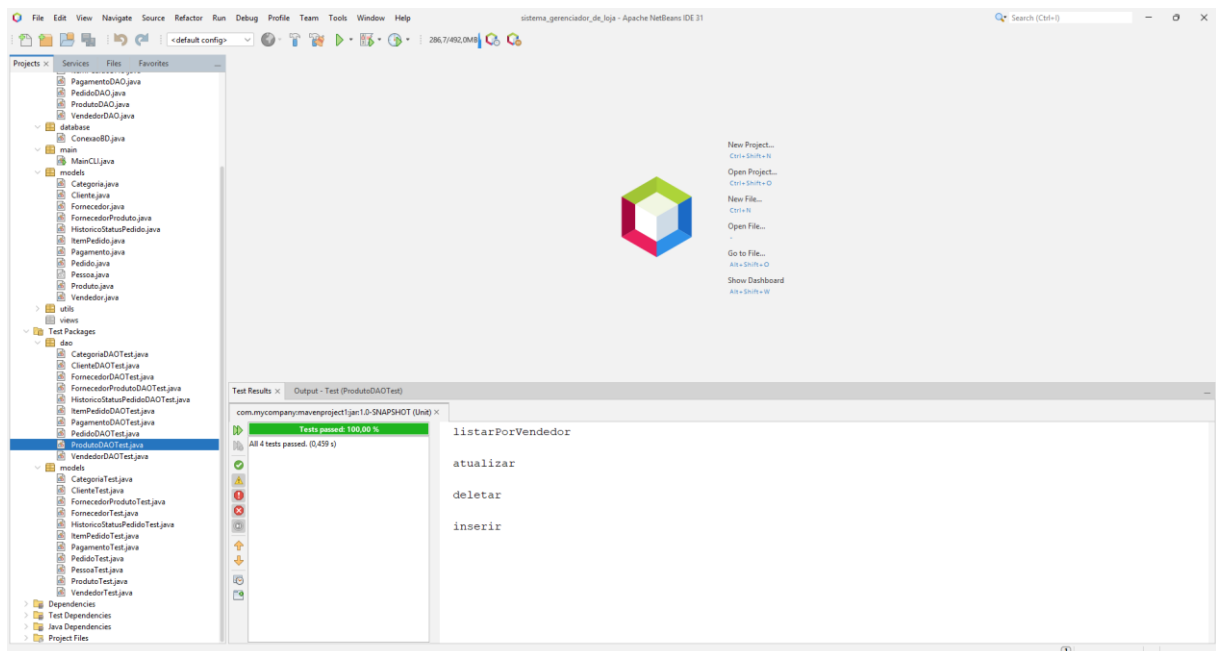
CT-DAO-03: Atualização de Produto

- **Identificador:** CT-DAO-03
- **Descrição:** Validar a alteração de dados cadastrais de um produto já persistido na base de dados (como nome, preço e estoque).
- **Passos para execução:**
 1. Inserir um produto inicial na base (ex: "Mouse Gamer").
 2. Alterar o nome para "Mouse Gamer Pro", o preço para 199.99 e o estoque para 15.
 3. Chamar `produtoDAO.atualizar(produto)`.
 4. Executar a listagem para confirmar as modificações.
- **Dados de Entrada:** Objeto Produto com os atributos modificados.
- **Saída Esperada:** Os dados do produto são atualizados com sucesso no registro correspondente.
- **Critério de Sucesso:** A listagem confirma que o nome, preço unitário e estoque refletem os novos valores atualizados.

CT-DAO-04: Deleção de Produto

- **Identificador:** CT-DAO-04

- **Descrição:** Validar a remoção de um produto específico da base de dados utilizando o ID do produto e o ID do vendedor.
- **Passos para execução:**
 1. Inserir um produto de teste (ex: "Teclado Mecânico").
 2. Obter o idProduto gerado.
 3. Chamar `produtoDAO.deletar(idProduto, vendedorTeste.getIdPessoa())`.
 4. Consultar a listagem do vendedor para verificar a exclusão.
- **Dados de Entrada:** ID do produto e ID do vendedor.
- **Saída Esperada:** O produto é completamente removido da base de dados.
- **Critério de Sucesso:** A lista de produtos retornada para o vendedor fica vazia (`produtos.isEmpty()`).



CLASSE VENDEDOR (MODELS E DAO)

1.10.1 Casos de Teste da Camada Models (VendedorTest)

CT-MOD-01: Definição de login válido

- **Identificador:** CT-MOD-01
- **Descrição:** Validar se o método `setLogin` aceita e armazena corretamente um login válido dentro dos limites permitidos.
- **Passos para execução:**
 1. Instanciar um objeto `Vendedor`.
 2. Chamar `instance.setLogin("paulo.costa")`.
- **Dados de Entrada:** "paulo.costa"
- **Saída Esperada:** O login do vendedor é atualizado com sucesso.

- **Critério de Sucesso:** `getLogin()` retorna "paulo.costa".

CT-MOD-02: Tentativa de definir login vazio

- **Identificador:** CT-MOD-02
- **Descrição:** Validar se o sistema rejeita a atribuição de uma string vazia como login.
- **Passos para execução:**
 1. Instanciar um objeto `Vendedor`.
 2. Chamar `instance.setLogin("")` dentro de uma asserção de exceção.
- **Dados de Entrada:** "" (string vazia)
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O login não pode ser nulo ou vazio."*
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-03: Tentativa de definir login nulo

- **Identificador:** CT-MOD-03
- **Descrição:** Validar se o sistema impede a definição de um login nulo.
- **Passos para execução:**
 1. Instanciar um objeto `Vendedor`.
 2. Chamar `instance.setLogin(null)` dentro de uma asserção de exceção.
- **Dados de Entrada:** null
- **Saída Esperada:** Lançamento de `IllegalArgumentException`.
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-04: Tentativa de definir login maior que 50 caracteres

- **Identificador:** CT-MOD-04
- **Descrição:** Validar se o sistema restringe o tamanho máximo do login a 50 caracteres.
- **Passos para execução:**
 1. Instanciar um objeto `Vendedor`.
 2. Gerar uma string com 51 caracteres (`"a".repeat(51)`).
 3. Chamar `instance.setLogin(loginGrande)` dentro de uma asserção de exceção.
- **Dados de Entrada:** String com 51 caracteres
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem *"O login deve ter no máximo 50 caracteres."*
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-05: Definição de login com limite exato de 50 caracteres

- **Identificador:** CT-MOD-05

- **Descrição:** Validar se o sistema aceita um login contendo exatamente o limite máximo permitido de 50 caracteres.
- **Passos para execução:**
 1. Instanciar um objeto Vendedor.
 2. Gerar uma string com 50 caracteres ("a".repeat(50)).
 3. Chamar instance.setLogin(login).
- **Dados de Entrada:** String com 50 caracteres
- **Saída Esperada:** O login é aceito e armazenado sem exceções.
- **Critério de Sucesso:** getLogin() retorna a string exata de 50 caracteres.

CT-MOD-06: Definição de senha hash válida

- **Identificador:** CT-MOD-06
- **Descrição:** Validar se o método setSenhaHash aceita e armazena corretamente uma hash de senha válida que atende ao requisito mínimo de tamanho.
- **Passos para execução:**
 1. Instanciar um objeto Vendedor.
 2. Chamar instance.setSenhaHash("hash862526").
- **Dados de Entrada:** "hash862526"
- **Saída Esperada:** A senha hash é atualizada com sucesso.
- **Critério de Sucesso:** getSenhaHash() retorna "hash862526".

CT-MOD-07: Tentativa de definir senha hash vazia

- **Identificador:** CT-MOD-07
- **Descrição:** Validar se o sistema rejeita a atribuição de uma string vazia como senha hash.
- **Passos para execução:**
 1. Instanciar um objeto Vendedor.
 2. Chamar instance.setSenhaHash("") dentro de uma asserção de exceção.
- **Dados de Entrada:** "" (string vazia)
- **Saída Esperada:** Lançamento de IllegalArgumentException com a mensagem *"A senha hash não pode ser nula ou vazia."*
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

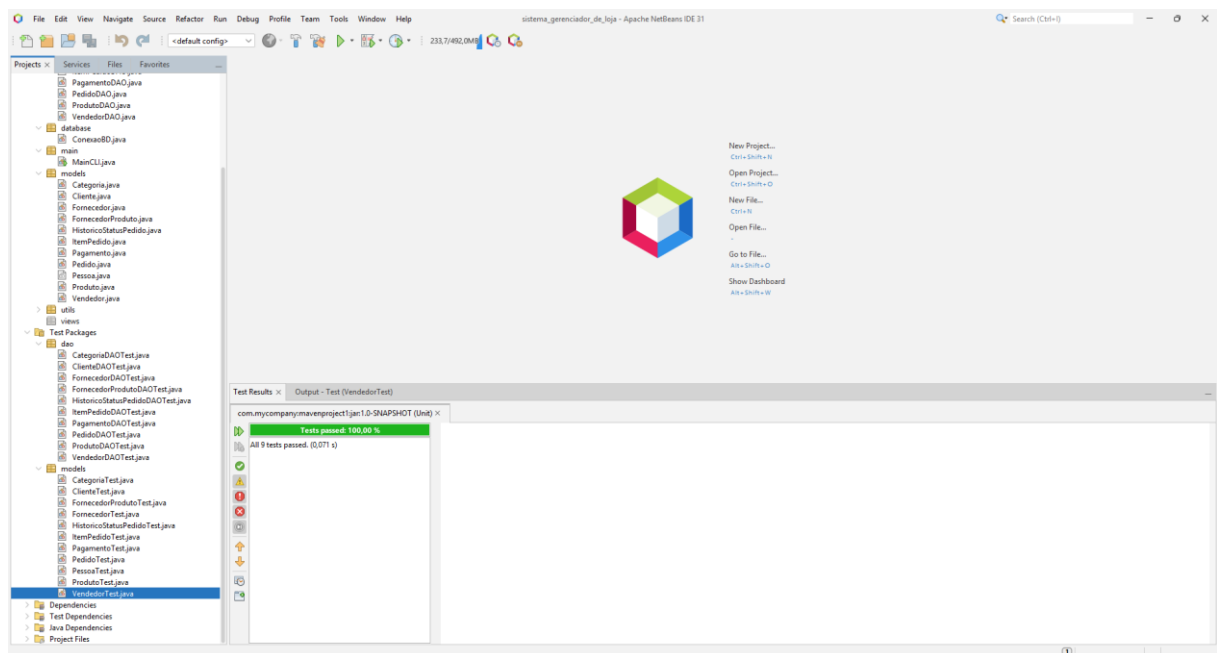
CT-MOD-08: Tentativa de definir senha hash nula

- **Identificador:** CT-MOD-08
- **Descrição:** Validar se o sistema impede a definição de uma senha hash nula.
- **Passos para execução:**

1. Instanciar um objeto Vendedor.
 2. Chamar `instance.setSenhaHash(null)` dentro de uma asserção de exceção.
- **Dados de Entrada:** null
 - **Saída Esperada:** Lançamento de `IllegalArgumentException`.
 - **Critério de Sucesso:** Exceção disparada e capturada adequadamente.

CT-MOD-09: Tentativa de definir senha hash menor que 6 caracteres

- **Identificador:** CT-MOD-09
- **Descrição:** Validar se o sistema rejeita senhas hash com menos de 6 caracteres por considerá-las inseguras.
- **Passos para execução:**
 1. Instanciar um objeto Vendedor.
 2. Chamar `instance.setSenhaHash("hash8")` dentro de uma asserção de exceção.
- **Dados de Entrada:** "hash8" (4 caracteres)
- **Saída Esperada:** Lançamento de `IllegalArgumentException` com a mensagem informando que a senha parece inválida ou insegura.
- **Critério de Sucesso:** Exceção disparada e capturada adequadamente.



1.10.2 Casos de Teste da Camada DAO (VendedorDAOTest)

CT-DAO-01: Inserção de Vendedor

- **Identificador:** CT-DAO-01
- **Descrição:** Validar a persistência transacional de um novo vendedor (envolvendo as tabelas pessoa e vendedor), a geração automática do ID correspondente (`id_pessoa`) e a recuperação via listagem.

- **Passos para execução:**
 1. Configurar o ambiente (setUp), limpando a base de dados.
 2. Instanciar um objeto Vendedor preenchendo os dados pessoais e credenciais de acesso.
 3. Chamar vendedorDAO.inserir(vendedor).
- **Dados de Entrada:** Objeto Vendedor preenchido com dados válidos.
- **Saída Esperada:** Registos inseridos com sucesso na base de dados utilizando transação (commit), gerando um ID válido.
- **Critério de Sucesso:** vendedor.getIdPessoa() > 0, a lista possui exatamente 1 registo e o nome corresponde ao inserido.

CT-DAO-02: Listagem de Vendedores

- **Identificador:** CT-DAO-02
- **Descrição:** Validar a recuperação de múltiplos vendedores cadastrados na base de dados através da listagem geral.
- **Passos para execução:**
 1. Inserir dois vendedores de teste distintos através do DAO.
 2. Chamar vendedorDAO.listar().
- **Dados de Entrada:** Nenhum (consulta geral).
- **Saída Esperada:** Retorno de uma lista contendo exatamente os dois vendedores inseridos.
- **Critério de Sucesso:** A lista não é nula e o tamanho retornado é igual a 2.

CT-DAO-03: Atualização de Vendedor

- **Identificador:** CT-DAO-03
- **Descrição:** Validar a alteração transacional de dados cadastrais de um vendedor já persistido (nome, email e login).
- **Passos para execução:**
 1. Inserir um vendedor inicial na base.
 2. Modificar o nome, email e login do objeto.
 3. Chamar vendedorDAO.atualizar(vendedor).
 4. Executar a listagem para confirmar as modificações.
- **Dados de Entrada:** Objeto Vendedor com os atributos modificados.
- **Saída Esperada:** Os dados nas tabelas pessoa e vendedor são atualizados com sucesso de forma atômica.

- **Critério de Sucesso:** A listagem confirma que o nome, email e login refletem os novos valores atualizados.

CT-DAO-04: Deleção de Vendedor

- **Identificador:** CT-DAO-04
- **Descrição:** Validar a remoção transacional de um vendedor específico e dos seus dados de pessoa associados.
- **Passos para execução:**
 1. Inserir um vendedor de teste.
 2. Obter o idVendedor.
 3. Chamar vendedorDAO.deletar(idVendedor).
 4. Consultar a listagem geral para verificar a exclusão.
- **Dados de Entrada:** ID do vendedor (id_pessoa).
- **Saída Esperada:** O vendedor e a respetiva pessoa são completamente removidos da base de dados.
- **Critério de Sucesso:** A lista de vendedores retornada fica vazia (lista.isEmpty()).

CT-DAO-05: Autenticação de Vendedor (Sucesso e Falhas)

- **Identificador:** CT-DAO-05
- **Descrição:** Validar o mecanismo de autenticação por login e senha, assegurando sucesso com credenciais válidas e retorno nulo para credenciais inválidas ou inexistentes.
- **Passos para execução:**
 1. Inserir um vendedor com login e senha hash conhecidos e status ativo.
 2. Chamar vendedorDAO.autenticar("fdias", "senhaSegura123") (caso de sucesso).
 3. Chamar vendedorDAO.autenticar("fdias", "senhaErrada") (falha por senha incorreta).
 4. Chamar vendedorDAO.autenticar("loginInexistente", "senhaSegura123") (falha por login inexistente).
- **Dados de Entrada:** Pares de strings (login e senha).
- **Saída Esperada:** Retorno do objeto Vendedor correspondente no caso de sucesso e null nos cenários de falha.
- **Critério de Sucesso:** O objeto autenticado não é nulo e contém os dados corretos; as tentativas inválidas retornam estritamente null.

